

FH Aachen
Fachbereich Elektrotechnik und Informationstechnik

BACHELOR'S THESIS

Hybrid Local-First E-Commerce

Reconciling Offline Capability with Strict Data Consistency

EXAMINERS

Prof. Dr. Andreas Hannig

M. Eng. Marcel Remmy

CANDIDATE

Simon Felix Conrad

Student Number: 3597903

Academic year 2025-2026

Erklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die im Literaturverzeichnis angegebenen Quellen und Hilfsmittel benutzt habe.

Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder noch nicht veröffentlichten Quellen entnommen sind, sind als solche kenntlich gemacht. Zudem sind die Stellen gekennzeichnet, die mit Hilfsmitteln erstellt wurden.

Die Zeichnungen oder Abbildungen in dieser Arbeit sind von mir selbst erstellt worden oder mit einem entsprechenden Quellen- oder Hilfsmittelnachweis versehen.

Diese Arbeit ist in gleicher oder ähnlicher Form noch bei keiner anderen Prüfungsbehörde eingereicht worden.

Aachen, im Monat Jahr

Abstract

The architectural evolution of mobile e-commerce has historically relied on server-centric models that remain vulnerable to network volatility and latency. While the “Local-First Software” paradigm offers a compelling alternative by relocating data and computational authority to the client device, its adoption in retail faces significant hurdles. Specifically, the “Invariant Problem” and strict financial regulations (e.g., PSD2) complicate this approach. Standard local-first data structures, such as Conflict-Free Replicated Data Types (CRDTs), excel at structural convergence but struggle to enforce the strict semantic consistency and server-authoritative gatekeeping required for rivalrous inventory and secure checkout.

This thesis investigates the architectural viability and performance trade-offs of applying a hybrid local-first paradigm to e-commerce. Following the Design Science Research (DSR) methodology, a “Three-Pillar” software artifact was designed and instantiated. By decomposing consistency requirements through the lens of the PACELC framework, the architecture applies a highly available, optimistic read-path for catalog browsing and collaborative cart management using locally-reactive replication (ElectricSQL) and CRDTs (Yjs). Conversely, it enforces strict serializability for checkout operations via a synchronous, server-authoritative RPC write-path (tRPC), unified by a Command-Query Responsibility Segregation (CQRS) reconciliation loop. The artifact was empirically benchmarked against an industry-standard SSR-Hydrated SPA across varying network conditions, hardware profiles, and dataset sizes.

The evaluation reveals a paradigm shift in systemic bottlenecks. The hybrid architecture neutralizes network latency during active navigation and provides robust offline autonomy, but relocates the computational burden heavily onto the client device’s CPU and memory. While the local-first approach outperforms the SSR-SPA baseline under degraded network conditions, it imposes a severe initial synchronization penalty and degrades performance on constrained hardware. Crucially, the CQRS-driven hybrid design resolves the tension between optimistic offline availability and transactional integrity, blocking semantic conflicts without requiring complex client-side formal verification.

Ultimately, the findings validate that a hybrid consistency model can reconcile local-first principles with the strict constraints of digital retail. However, due to the high initial data payload and reliance on client-side processing, this architecture is currently most viable for B2B commerce and industrial field operations, offering a pragmatic compromise between seamless edge responsiveness and authoritative transactional security.

Keywords: Local-First, E-Commerce, Distributed Systems, CRDT, PACELC, CQRS, DSR

Table of Contents

List of Figures	xii
List of Tables	xiv
List of Abbreviations	xvi
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Research Objective and Methodology	2
1.4 Thesis Structure	2
2 State of Research	5
2.1 The Baseline of Retail Consistency	5
2.1.1 Amazon Dynamo and High-Availability Carts	5
2.1.2 The Checkout Conflict and Sagas	5
2.2 Academic Approaches to Local-First Invariants	6
2.2.1 ConLoc (Static Verification)	6
2.2.2 LoRe (Reactive Programming Verification)	6
2.2.3 The Escrow Pattern Approach	7
2.3 The Identified Research Gap	7
2.4 Conclusion	8
3 Theory	11
3.1 The Landscape of Distributed Data Systems	11
3.2 Distributed Systems Constraints (CAP & PACELC)	11
3.2.1 The CAP Theorem: A Foundation for Trade-offs	11
3.2.2 The PACELC Framework: A More Complete Model	12
3.2.3 Implications for Modern Application Architectures	12
3.3 The Local-First Software Paradigm	13
3.3.1 Motivation: Reconciling Collaboration and Ownership	13
3.3.2 Architectural Shift: From Server-Authoritative to Client-Authoritative State	13
3.3.3 The Seven Ideals of Local-First Software	14
3.4 Core Technology: CRDTs	15
3.4.1 Principle of Strong Eventual Consistency	15
3.4.2 Beyond Convergence: Causal Consistency	16

3.4.3	Role in Local-First Collaboration	16
3.5	Advanced Challenges in Local-First Architectures	16
3.5.1	Semantic Conflicts and Invariant Preservation	16
3.6	Technical Mechanisms of Persistence and UI	17
3.6.1	Mechanics of Optimistic UI and Speculative State	17
3.6.2	The Complexity of Divergence and Rollback	18
3.6.3	Command-Query Responsibility Segregation (CQRS)	18
3.7	Regulatory and Domain Constraints in E-Commerce	19
3.8	Theoretical Frameworks for System Evaluation	19
3.8.1	Defining Architectural Viability and Correctness	19
3.8.2	The Theory of Comparative Benchmarking	20
4	Methodology	23
4.1	Research Design: Design Science	23
4.1.1	The DSRM Process Model: A Roadmap	23
4.1.2	The Three Cycles of Inquiry	24
4.2	Domain Analysis: The Theoretical Framework	25
4.2.1	The Consistency Dilemma and PACELC	25
4.2.2	The Decomposition Matrix	25
4.2.3	Methodological Rationale for the Collaborative Cart	26
4.2.4	Addressing Authoritative Gatekeeping	27
4.3	Derivation of Design Requirements (Conceptual Framework)	27
4.3.1	Requirement 1: The Read-Path (Offline-Available Catalog)	27
4.3.2	Requirement 2: The Collaborative-Path (Mathematical Convergence)	28
4.3.3	Requirement 3: The Write-Path (Synchronous Authority)	28
4.3.4	Requirement 4: The Feedback Loop (State Reconciliation)	29
4.4	Artifact Instantiation	29
4.4.1	Comparative Analysis and Selection Rationale	29
4.4.2	Architecture Synthesis	31
4.5	Evaluation Methodology	31
4.5.1	The SSR-Hydrated SPA Control Group	32
4.5.2	Statistical Approach and Dataset Scaling	33
4.5.3	Performance and Resource Utilization	34
4.5.4	Network Efficiency	34
4.5.5	Distributed State Consistency and Conflict Resolution	35
4.5.6	Offline Feature Availability	36
4.6	Limitations and Scope	36
5	Results	39
5.1	The “Three-Pillar” Hybrid Architecture	39

5.1.1	Architectural Components and Data Flow	39
5.2	Pillar I: The Optimistic Read Layer (Catalog Management)	41
5.2.1	Shape-Based Replication	41
5.2.2	Reactive Query Engine	41
5.2.3	Offline Application Shell and Asset Caching	42
5.3	Pillar II: The Collaborative State (The Shopping Cart)	43
5.3.1	The CRDT Data Model	44
5.3.2	Network Adapter Configuration and Transport Consolidation	44
5.4	Pillar III: The Authoritative Gate (Checkout & Inventory)	44
5.4.1	The Synchronous Write Path (tRPC)	45
5.4.2	Snapshotting for Historical Integrity	45
5.5	The Feedback Loop: Reactive Reconciliation	45
5.5.1	The Command-Query Cycle and Write-Path Synchronization	45
6	Evaluation	49
6.1	Performance Benchmarks	49
6.1.1	Initial Synchronization and Load Latency	49
6.1.2	Interaction Latency	50
6.2	Client Resource Utilization	51
6.2.1	Browser Storage Footprint	52
6.2.2	Client Memory Tax	52
6.3	Network Utilization and Efficiency	53
6.3.1	Bandwidth Consumption During Navigation	53
6.3.2	Delta Update Payload Efficiency	54
6.4	Distributed State Consistency and Conflict Resolution	54
6.4.1	Conflict Scenario A: Execution and Gatekeeping	54
6.4.2	Conflict Scenario B: Structural Concurrency	55
6.4.3	Time to Consistency (TTC)	55
6.4.4	Server-Authoritative Gatekeeping (Malicious Payload Interception)	56
6.5	Offline Feature Availability	56
7	Discussion	59
7.1	Navigating the PACELC Trade-offs: Latency vs. Client Resources	59
7.2	Resolving the Invariant Problem and Authoritative Gatekeeping	60
7.3	The Local-First Spectrum: Adapting Ideals to E-Commerce	61
7.4	Structural Concurrency and Collaborative Resilience	62
7.5	Network Utilization and Offline Functionality	62
7.6	Limitations	62
7.7	Conclusion	63
8	Outlook	65

8.1	Industry Adoption and Market Viability	65
8.2	Functional Extensions and Conflict Resolution	66
8.3	Avenues for Future Research	67
9	Conclusion	69
9.1	Key Findings and Contributions	69
9.1.1	The Paradigm of Shifting Bottlenecks: Network vs. Client Resources	69
9.1.2	Resolving Invariant and Regulatory Constraints through CQRS	70
9.1.3	The Local-First Spectrum in Practice	70
9.1.4	Network Efficiency and Offline Autonomy	70
9.2	Implications for Software Engineering and E-Commerce	70
9.3	Final Remarks	71
	Bibliography	73
	Einsatz generativer KI	77
A	Raw Benchmark Data	81
A-A	Initial Synchronization and Load Latency	81
A-B	Interaction Latency (Search & Filter)	81
A-C	Browser Storage Footprint	82
A-D	Client Memory Tax (JS Heap)	83
B	Implementation Details	87
B-A	Solving Schema Rigidity with Custom Fields	87
B-B	Sorting via Fractional Indexing	87
B-C	Session Isolation and Reactivity	87
B-D	Awareness and Presence	87
B-E	Eliminating Render Flicker via Route Preloading	88
B-F	Self-Healing Orphaned Nodes	89

List of Figures

Figure 5.1	Architecture Diagram showing the data flow between Client, Sync Engine, and Authoritative Server	40
Figure 5.2	The Partslist search page interface, demonstrating instantaneous local filtering and sorting driven by the reactive query engine.	42
Figure 5.3	The Command-Query Cycle and Write-Path Synchronization	46
Figure 6.1	Initial Synchronization and Load Latency comparing Local-First vs SSR-SPA across varying dataset sizes. Error bars denote the 5th and 95th Percentile (P95).	49
Figure 6.2	Median Interaction Latency comparing Local-First vs SSR-SPA under Baseline and Commuter network profiles for the Mid-Sized Dataset.	50
Figure 6.3	Median Interaction Latency comparing Local-First vs SSR-SPA under Budget and Worst-Case profiles for the Large Retailer Dataset.	51
Figure 6.4	Browser Storage Footprint comparing Local-First vs SSR-SPA across varying dataset sizes.	52
Figure 6.5	Client Memory Tax (JS Heap) during the intensive initial synchronization and rendering of the Large Retailer dataset.	52
Figure 6.6	Cumulative Bandwidth Consumption during a typical user journey (Homepage -> Search -> Product) with the Mid-Sized Store.	53

List of Tables

Table 4.1 Mapping the DSRM to Chapter Structure	24
Table 4.2 Decomposition Matrix of E-Commerce Consistency Requirements	26

List of Abbreviations

3DS2:	3D Secure 2.0
ACID:	A tomicity, C onsistency, I solation, D urability
AI:	A rtificial I ntelligence
API:	A pplication P rogramming I nterface
APM:	A lternative P ayment M ethod
B2B:	B usiness- to - B usiness
B2C:	B usiness- to - C onsumer
CDc:	C hange D ata C apture
CDP:	C hrome D evTools P rotocol
ConLoc:	C onsistent L ocal-First Software
CPU:	C entral P rocessing U nit
CQRS:	C ommand- Q uery R esponsibility S egregation
CRDT:	C onflict-Free R eplicated D ata T ype
CSS:	C ascading S tyle S heets
CVV:	C ard V erification V alue
DB:	D atabase
DOM:	D ocument O bject M odel
DSR:	D esign S cience R esearch
DSRM:	D esign S cience R esearch M ethodology
EAV:	E ntity- A tttribute- V alue
FRP:	F unctional R eactive P rogramming
GDPR:	G eneral D ata P rotection R egulation
HTML:	H yper T ext M arkup L anguage

HTTP:	H yper T ext T ransfer P rotocol
JML:	J ava M odeling L anguage
JS:	J ava S cript
JSON:	J ava S cript O bject N otation
LCP:	L argest C ontentful P aint
LWW:	L ast- W rite- W ins
MIT:	M erchant- I nited T ransaction
NoSQL:	N ot O nly S QL
OOM:	O ut of M emory
ORM:	O bject- R elational M apping
P₉₅:	95 th P ercentile
PA/EL:	P artition- A vailability/ E lse- L atency
PAN:	P rimary A ccount N umber
PC/EC:	P artition- C onsistency/ E lse- C onsistency
PCI-DSS:	P ayment C ard I ndustry D ata S ecurity S tandard
PSD2:	European Union's Revised P ayment S ervice D irective
REST:	R Epresentational S tate T ransfer
RPC:	R emote P rocedure C all
RTT:	R ound T rip T ime
SAD:	S ensitive A uthentication D ata
SCA:	S trong C ustomer A uthentication
SDL:	S chema D efinition L anguage
SEC:	S trong E ventual C onsistency
SKU:	S tock K eeping U nit
SMT:	S atisfiability M odulo T heories

SPA:	S ingle P age A pplication
SQL:	S tructured Q uery L anguage
SSR:	S erver- S ide R endering
tsRPC:	T ypeScript R emote P rocedure C all
TTC:	T ime to C onsistency
TTFB:	T ime to F irst B yte
TTI:	T ime to I nteractive
UI:	U ser I nterface
UX:	U ser E xperience
Wasm:	W eb A ssembly
WebRTC:	W eb R eal- T ime C ommunication
YCSB:	Y ahoo! C loud S erving B enchmark

Introduction

1.1 Background and Motivation

The architectural evolution of e-commerce platforms has historically gravitated towards a server-centric paradigm. In this model, the server acts as the absolute source of truth and the primary executor of business logic, while the client device operates as a thin rendering layer. However, this architecture is predicated on the assumption of continuous connectivity, falling prey to the “fallacy of the reliable network”. Mobile connectivity is inherently volatile, often plagued by intermittent signal loss, high latency, and network partitions [1], [2], [3]. For example, intermittent 5G network disconnections can trigger an average data throughput loss of 60% [4]. Furthermore, this connection volatility can cause severe unreliability, with median packet loss rates exceeding 50% for certain operators during data-intensive mobile tasks [4].

In the highly competitive domain of digital retail, network latency translates directly to economic loss. Empirical research demonstrates that even marginal increases in waiting times lead to user dissatisfaction and decreased conversion rates [5], [6, chap. 1]. For instance, an increase in latency of just 100 milliseconds has been shown to decrease conversion rates by 2.4% to 7.1% [5]. Conversely, users routed to a high-performance website are 15% more likely to complete a purchase [5]. Traditional cloud-based applications, operating under the constraints of distributed system trade-offs like the PACELC theorem [7], often struggle to mask these connectivity issues, frequently presenting users with loading spinners or completely locking them out of their tasks when offline.

A compelling approach to mitigating this broader class of network dependency is the “Local-First Software” paradigm [1]. Originally formulated to provide seamless offline capabilities and robust data ownership in collaborative, document-based applications, this architecture fundamentally relocates the primary copy of data and application logic to the user’s local device. By decoupling interface responsiveness from network quality, local-first principles ensure high availability and zero-latency interactions. While this paradigm has seen successful adoption in its native context, often leveraging **Conflict-Free Replicated Data Types** (CRDTs) [8], its application to the intricate, constraint-heavy, and highly transactional environment of e-commerce remains a complex [3], unresolved challenge [9].

1.2 Problem Statement

The core tension in local-first e-commerce lies in reconciling the optimistic, highly available nature of local data replication with the strict, relational data integrity required for retail transactions. This is fundamentally an issue of distributed consistency.

Standard CRDTs excel at structural convergence but fail to enforce semantic validity, a phenomenon known as the “Invariant Problem” [9]. For instance, if a local-first application

allows two disconnected users to optimistically check out the last remaining unit of a product, the subsequent background synchronization will result in a negative inventory balance. In e-commerce, certain financial operations and rivalrous goods strictly require serializability and authoritative gatekeeping to prevent invalid states, which contradicts the local-first ideal of total offline availability.

Compounding this mathematical “Invariant Problem” is a strict regulatory barrier: European financial compliance frameworks, such as European Union's Revised **Payment Service Directive** (PSD2) and its mandate for **Strong Customer Authentication** (SCA), severely impede the fully offline processing of electronic payments, rendering a purely decentralized, synchronous checkout process unviable for standard retail.

The research problem is therefore defined by the following question: *To what extent does the application of the local-first paradigm to e-commerce platforms require a hybrid architecture to reconcile offline capability with strict data consistency, and what are the specific trade-offs involved?*

1.3 Research Objective and Methodology

The primary objective of this thesis is to investigate the architectural viability and performance implications of a “hybrid local-first” paradigm within a relational e-commerce context.

To address this engineering challenge, this research adopts the **Design Science Research** (DSR) methodology as defined by Hevner et al. [10] and Peffers et al. [11].

This study seeks to design, instantiate, and evaluate a functional “Three-Pillar” hybrid architecture. This architecture aims to operationalize a decomposition of consistency requirements: applying a highly available, low-latency model for catalog browsing and collaborative cart management, while simultaneously enforcing a strongly consistent, server-authoritative mechanism for checkout transactions. By benchmarking this artifact against an industry-standard **Server-Side Rendering** (SSR)-Hydrated **Single Page Application** (SPA) baseline, this research quantifies the resulting trade-offs concerning interaction latency, computational resource utilization, and offline functional resilience.

1.4 Thesis Structure

The remainder of this thesis is structured as follows:

- **State of Research (Chapter 2):** Reviews the current academic and industrial landscape, analyzing the limitations of Optimistic **User Interface** (UI) patterns and the “Invariant Problem” in distributed retail.
- **Theory (Chapter 3):** Establishes the foundational principles of distributed data systems, exploring the CAP theorem, the PACELC framework, and the theoretical underpinnings of Local-First Software and CRDTs.
- **Methodology (Chapter 4):** Details the Design Science Research methodology, operationalizing the theoretical framework into concrete design requirements for a hybrid architecture.

- **Results (Chapter 5):** Answers the first half of the research question by describing the technical instantiation of the “Three-Pillar” artifact, detailing the integration of partial replication (ElectricSQL), reactive local databases (TanStack DB), conflict-free collaboration (Yjs) and synchronous authoritative write paths (tRPC).
- **Evaluation (Chapter 6):** Answers the second half of the research question by presenting the empirical data gathered from the laboratory simulations, comparing the local-first artifact against an SSR-Hydrated SPA baseline across various network and hardware constraints.
- **Discussion (Chapter 7):** Interprets the empirical findings, assessing how the architecture navigates the trade-offs between network latency, client-side resource taxation, and strict business invariants.
- **Outlook (Chapter 8):** Explores the broader industry viability of local-first e-commerce, specifically in high-latency or **Business-to-Business** (B2B) markets, and proposes future architectural extensions.
- **Conclusion (Chapter 9):** Summarizes the primary contributions of the research and its implications for modern web engineering.

State of Research

Building upon the theoretical framework of distributed constraints, this chapter examines the current academic and industrial landscape of decentralized consistency within retail systems. E-commerce architectures currently face a complex trade-off: balancing the high-availability demands of modern user experiences with the strict data integrity required for transactional correctness.

While the “Local-First” movement has successfully inverted the traditional client-server hierarchy for collaborative tools [1], applying these autonomous principles to complex, constraint-heavy retail environments remains an area of active investigation [9], [12].

This chapter evaluates foundational industry models like Amazon’s Dynamo, assesses emerging academic solutions that attempt to enforce local-first invariants, and synthesizes the remaining research gaps that prevent widespread adoption in modern e-commerce.

2.1 The Baseline of Retail Consistency

To understand the challenge of local-first retail, it is necessary to analyze prevailing server-side consistency models. A foundational text in modern distributed retail is the Amazon Dynamo paper, which established the blueprint for the “always-on” shopping cart experience [13].

2.1.1 Amazon Dynamo and High-Availability Carts

Amazon’s Dynamo prioritized write availability, driven by the economic reality that rejecting an “Add to Cart” action due to server unavailability results in direct revenue loss [13].

To achieve this, Dynamo relies on eventual consistency, where divergent replicas (such as two versions of a shopping cart existing on different server nodes due to a network partition) are reconciled post-hoc via vector clocks. In the context of the PACELC theorem (discussed in Chapter 3), large-scale retail systems modeled after Dynamo generally operate in the PA/EL quadrant, explicitly prioritizing Availability and Latency over Strong Consistency [7].

The primary finding from the Dynamo architecture is that eventual consistency is highly effective for additive, user-specific data (like a cart). However, despite its robustness against internal server node failures, Dynamo assumes a reliable connection from the client to the edge. It does not account for terminal connectivity issues at the client device, meaning server-side availability cannot be conflated with true end-to-end system reliability in disconnected mobile environments.

2.1.2 The Checkout Conflict and Sagas

While the eventual consistency model accommodates the highly available shopping cart, literature agrees it is insufficient for the checkout phase. While temporary inconsistency in

a cart is often acceptable, the checkout process involves strict invariants (e.g., `stock >= 0`) that cannot be violated [14].

The consensus in the research is that a “one-size-fits-all” consistency model is inadequate; retail platforms often adopt a hybrid approach, switching between AP (Eventual) models for discovery and CP (Strong) models for transaction finality [13].

2.2 Academic Approaches to Local-First Invariants

As the industry recognized the limitations of pure server-side availability, academic research began exploring how to enforce strict invariants on client-replicated data without requiring immediate central coordination. Two prominent recent systems, ConLoc and LoRe, attempt to formalize this mixed-consistency management.

2.2.1 ConLoc (Static Verification)

Consistent Local-First Software (ConLoc) [9] targets object-oriented, imperative programming environments (specifically Java) to enforce application-level invariants on replicated data. Its primary goal is to statically analyze code to distinguish between operations that are safe for local execution and those that require network coordination.

To achieve this, developers must annotate class methods with pre- and post-conditions using the **Java Modeling Language (JML)** to define valid state transitions. ConLoc utilizes the **Z3 Satisfiability Modulo Theories (SMT)** solver to statically verify these annotations against two criteria: “Invariant Sufficiency” (sequential correctness) and “Mergeability” (concurrent correctness). Based on this analysis, operations are automatically tagged:

- **Weak:** Operations that preserve invariants under concurrent execution are allowed to execute locally.
- **Strong:** Operations that risk violation trigger automatic coordination via the Zookeeper Atomic Broadcast protocol.

While ConLoc successfully synthesizes correct synchronization logic, its reliance on JML annotations limits its flexibility in standard web development contexts.

2.2.2 LoRe (Reactive Programming Verification)

LoRe [12] takes a different approach by proposing a domain-specific language based on **Functional Reactive Programming (FRP)**. Rather than retrofitting imperative code, LoRe employs a verifying compiler to mathematically guarantee that specific data flows and user actions are “correct-by-construction.”

The system models data as “Reactives” and user actions as “Interactions”. Unlike ConLoc’s object-oriented approach, LoRe compiles code to the Viper verification infrastructure to check for invariant preservation (local safety) and confluence (concurrent safety). When the compiler detects that two interactions are non-confluent, meaning their concurrent execution could violate an invariant, it automatically generates a token-based distributed locking protocol to serialize their execution.

Although LoRe provides strong mathematical safety guarantees, it imposes a high “specification burden.” The requirement for developers to define invariants in first-order logic, combined with the potential for reduced availability due to aggressive locking, serves as a significant barrier to adoption in the fast-moving retail sector.

2.2.3 The Escrow Pattern Approach

To address numerical invariants without strict locking, researchers have also evaluated “Bounded CRDTs” utilizing the Escrow or Reservation pattern [15], [16]. In this model, the right to decrement stock is pre-allocated to clients.

While conceptually promising, findings indicate the Escrow pattern shifts the consistency problem to a “rights management” problem that introduces significant inventory fragmentation. By pre-allocating stock “rights” to client devices to enable offline purchasing, that inventory is effectively “locked” and unavailable to other online customers. If an offline user fails to complete their checkout, the stock remains in limbo until the lease expires, creating an unacceptable economic tension between offline availability and overall inventory efficiency [14], [15].

2.3 The Identified Research Gap

Despite the maturity of local-first principles for collaborative software and the scalability of Dynamo-like systems for the cloud, a significant gap exists in applying offline-ready architectures to practical retail. This thesis identifies three specific areas that require further development: the focus on sequential data types, the “Invariant Problem” and “Trust Boundaries”:

The “Text Editor Bias” and Relational Complexity

Foundational research into CRDTs has largely focused on Sequence CRDTs (e.g., RGA, LWW-Register) designed to preserve user intention in unstructured documents [9], [17]. In these scenarios, convergence is the primary goal. However, retail data is highly structured and constrained. The “intention” to purchase an item is not merely a data mutation but a claim on a rivalrous resource. Merging relational data in a peer-to-peer setting introduces complexities distinct from sequence CRDTs:

- **Referential Integrity:** If Node A creates an Order for Product X, while Node B concurrently deletes Product X, an uncoordinated merge may result in an orphan record, potentially impacting database integrity [9].
- **Schema Evolution:** Managing schema changes (migrations) across disconnected devices remains an open research topic [1].

Current research lacks a “Relational CRDT” framework capable of enforcing these constraints [9], [17].

The Decentralized Authority Gap

Existing local-first research often assumes a trusted client environment (e.g., collaborators working on a shared design file). E-commerce, however, introduces a fundamental conflict of interest: treating an inherently untrusted client device as the “Source of Truth” creates

a critical Authority Gap. Because a malicious user could attempt to alter prices or bypass inventory limits, the system must prove to the server that offline state transitions adhered to valid, replicated business logic (e.g., a discount code applied according to server-side rules) without requiring a full, synchronous re-execution of the transaction history. Current research lacks a standardized framework for Verifiable CRDTs that can cryptographically guarantee that offline mutations respect these server-defined invariants. [18]

The Regulatory Blindspot

Academic systems focused on data consistency fundamentally ignore the domain-specific legal realities of e-commerce. As detailed in the Theory chapter, regulations like PSD2 and **P**ayment **C**ard **I**ndustry **D**ata **S**ecurity **S**tandard (PCI-DSS) impede fully offline, synchronous payment authorization. Any local-first retail architecture that naively assumes all transactions can be fully finalized while disconnected is legally unviable.

2.4 Conclusion

The state of research indicates a chasm between the predictive, often fragile models of Optimistic UI currently used in industry, and the highly complex, formally verified mixed-consistency models proposed by academia. There is a lack of research investigating how pragmatic **C**ommand-**Q**uery **R**esponsibility **S**egregation (CQRS) patterns can be adapted for the local-first client edge to manage the latency and consistency requirements of e-commerce, while strictly adhering to regulatory boundaries. Addressing this specific architectural gap is the primary focus of this thesis.

Theory

This chapter establishes the theoretical foundations required to understand the complexities of local-first e-commerce. It explores the inherent constraints of distributed data systems, specifically analyzing the trade-offs dictated by the CAP theorem and the PACELC framework. Furthermore, it introduces the core tenets of the local-first software paradigm and details the enabling mechanisms of CRDTs. It addresses the advanced challenges of decentralized architectures, highlighting the friction between mathematical state convergence and semantic business invariants. Finally, this chapter outlines the technical mechanisms of modern state management, the regulatory constraints governing e-commerce, and the theoretical frameworks used to evaluate architectural viability and system performance.

3.1 The Landscape of Distributed Data Systems

The construction of modern distributed data systems requires navigating a complex landscape of fundamental trade-offs, primarily between data consistency, system availability, and operational performance [7]. The inherent challenges of coordinating data across multiple, geographically dispersed nodes have given rise to distinct architectural paradigms, each representing a different set of design choices. This section explores the core theoretical models that govern these trade-offs, providing a formal framework for understanding the design of contemporary distributed applications and setting the stage for the emergence of the local-first paradigm.

3.2 Distributed Systems Constraints (CAP & PACELC)

The local-first approach is fundamentally shaped by the environmental constraints of distributed systems, particularly the limitations of mobile networks. [1]

3.2.1 The CAP Theorem: A Foundation for Trade-offs

The CAP theorem [19] is a foundational theorem in distributed systems design, asserting that any distributed data store can only provide two of the following three guarantees:

- **Consistency (C):** Every operation appears to take effect instantaneously at a single point in time, and all nodes observe the same total order of operations. This property, formally known as linearizability (strong consistency), ensures the system behaves as if it were a single, non-replicated machine.
- **Availability (A):** Every request receives a (non-error) response, without the guarantee that it contains the most recent write. The system remains operational for reads and writes despite node failures.

- **Partition Tolerance (*P*):** The system continues to operate despite an arbitrary number of messages being dropped or delayed by the network between nodes. In the context of mobile computing, partitions are an unavoidable fact of the environment, not a design choice. [20]

Contrary to a common misconception that designers must choose two of these three properties for all operating conditions, the theorem’s core assertion, as articulated by D. Abadi [7], is more nuanced: a distributed system must choose between consistency and availability **only in the event of a network partition**. During normal operation, when the network is fully connected, the CAP theorem does not impose any limitations. A system can, in principle, deliver both full consistency and high availability when no partitions are present. The trade-off is forced only by the failure condition of a network partition.

3.2.2 The PACELC Framework: A More Complete Model

To provide a more comprehensive model of the choices faced by system designers, D. Abadi proposed the PACELC framework [7]. This model extends the CAP theorem by explicitly accounting for the trade-offs that exist during normal operation, when no network partition is present. The framework is structured as a conditional statement:

- **If there is a Partition (*P*)**, the system must make a trade-off between **Availability (*A*)** and **Consistency (*C*)**. This is the classic CAP theorem choice.
- **Else (*E*)**, during normal operation, the system must make a trade-off between **Latency (*L*)** and **Consistency (*C*)**.

The second part of the framework is crucial for understanding the design of modern, highly available systems. The requirement for high availability necessitates data replication; if data exists on only one node, the failure of that node makes the data unavailable. As Abadi notes, while the **occurrence** of a failure triggers the CAP trade-offs, the mere **possibility of failure** mandates replication during normal operation, which in turn creates the latency-consistency trade-off. To ensure strong consistency, an update must be synchronously propagated and confirmed across multiple replicas, a process that introduces significant latency. To reduce latency, a system can acknowledge a write asynchronously before it has been fully replicated, but this sacrifices strong consistency, as a read request might be served by a replica that has not yet received the latest update.

3.2.3 Implications for Modern Application Architectures

The PACELC framework clarifies the design philosophies behind many modern distributed applications. For example, systems like Amazon’s Dynamo [13] are classified as **PA/EL**: in the event of a partition (*P*), they are architected to preserve Availability (*A*) at the expense of strong Consistency (*C*), and during normal operation (*E*), they prioritize low Latency (*L*) over strong Consistency (*C*). In contrast, Traditional strong-consistency CP systems (often those prioritizing ACID semantics) are **PC/EC**: they refuse to sacrifice

consistency, accepting the cost of reduced availability during partitions and higher latency during normal operation.

These fundamental trade-offs left a significant gap in the design space for applications seeking both the low latency of client-side operations and the high availability required for modern workflows, especially in offline contexts [1]. This has motivated the development of novel architectural paradigms. Local-first software, in particular, emerges as a specific choice within the PACELC space, one that aggressively prioritizes Availability (A) and low Latency (L) for the end-user experience by fundamentally relocating the “primary” copy of data to the client device [1]. This architectural decision navigates the PACELC trade-offs in a new and user-centric manner.

3.3 The Local-First Software Paradigm

Local-first software represents a modern architectural paradigm designed to overcome the limitations of purely cloud-based applications. As framed by Kleppmann et al. [1], it aims to synthesize the best of two worlds: the seamless, real-time collaboration that defines cloud services and the performance, data ownership, and agency characteristic of traditional desktop software. By treating the data on a user’s local device as the primary copy, this model re-architects the relationship between the user, their data, and the network.

3.3.1 Motivation: Reconciling Collaboration and Ownership

The motivation for local-first software stems from a central tension in modern computing. Cloud applications have revolutionized collaboration, yet this convenience comes at a cost: by centralizing data, they strip users of “agency” and true ownership [1], [6, chap. 12]. For creative professionals who often have a “deep emotional attachment” to their work, this loss of control is a significant drawback. “Ownership,” in this context, refers not to intellectual property but to the user’s control over their data, free from a service provider’s mediation.

Conversely, traditional applications that store data in local files grant users complete ownership and control but have historically struggled with collaboration, often resulting in conflicting copies and tedious manual merging [21]. Local-first architecture seeks to resolve this tension by designing systems that deliver robust collaboration without forcing users to surrender ownership of their data.

3.3.2 Architectural Shift: From Server-Authoritative to Client-Authoritative State

To understand the mechanics of local-first e-commerce, one must first distinguish it from the dominant architectural patterns of the last decade. The transition to local-first software represents a fundamental inversion of responsibilities, shifting not merely the presentation layer, but also the data sovereignty and business logic from the cloud to the edge device. [1]

The Traditional Web (Server-Authoritative)

In standard cloud-based applications, the server retains exclusive custody of the application state and business logic. The client, typically a browser or a “thin” mobile app, acts

primarily as a remote rendering engine. As noted by Kleppmann et al. [1], the data on the server is treated as the primary, authoritative copy; if the client holds any data, it is merely a volatile cache subordinate to the server. Consequently, user interaction is strictly bound to the synchronous request/response cycle, where every state-changing operation requires a network round-trip [3]. In this model, the trust boundary is clearly demarcated at the **Application Programming Interface (API)** edge; the server protects the database and validates all incoming requests against a single, linear history [6, chap. 2].

Local-First (Client-Authoritative)

Local-first software leverages modern advancements in mobile computing power and embedded storage (e.g., SQLite, IndexedDB) to “onload” computation and data management back to the end-user device. In this architecture, the client is upgraded from a passive viewer to an active node in the distributed system, hosting a local database and the logic required to manipulate it. This allows the client to perform read and write operations independently, treating the network as an optional utility for eventual synchronization rather than a prerequisite for basic availability. [1]

3.3.3 The Seven Ideals of Local-First Software

Kleppmann et al. [1] define the local-first paradigm through seven distinct ideals. These principles collectively describe a user experience that prioritizes speed, reliability, collaboration, and user agency.

1. **No Spinners: Your Work at Your Fingertips** By treating the local copy of data as the primary authority, most user operations can be executed without a network round-trip. This architectural choice fundamentally eliminates network latency from the user’s critical path [22], enabling an interface that can respond nearly instantaneously to input. The goal is to create a tangible sense of speed and responsiveness, where the user never has to wait for a loading spinner to complete a core task.
2. **Your Work Is Not Trapped on One Device** Modern workflows are inherently multi-device. A user might start a task on a smartphone and finish on a laptop. Local-first software must therefore support the seamless synchronization of data across all of a user’s devices. While each device holds a primary copy of the data, a background synchronization process ensures that changes made on one device are propagated to all others.
3. **The Network Is Optional** A core tenet of the local-first paradigm is that the application must be fully functional even with an intermittent or non-existent network connection. Because data and application logic reside locally, users can continue their work unimpeded while offline. Changes are stored locally and are automatically synchronized when a network connection is restored, providing a level of resilience that is difficult to achieve in server-centric cloud applications.
4. **Seamless Collaboration with Your Colleagues** The ideal is to support multi-user, real-time collaboration that is at least on par with the best cloud applications. This capability transcends the conflict-resolution models of simple file-synchronization services like Dropbox, which often create “conflicted copies”. Local-first

collaboration requires underlying data structures that can automatically and intelligently merge changes from multiple collaborators, even when those changes were made offline.

5. **The Long Now** This ideal concerns the long-term preservation of a user’s work. In a cloud-centric model, data is trapped in a proprietary service; if the company fails, the data may become inaccessible, risking what Vint Cerf calls a “digital Dark Age” [23]. By storing data in local, user-controlled files, local-first software ensures that created works can remain accessible indefinitely, independent of the original software vendor.
6. **Security and Privacy by Default** Centralizing all user data on a single server creates an attractive target for attackers. The local-first model inherently enhances security and privacy by avoiding large, centralized databases. This architecture is naturally suited for end-to-end encryption, where data is encrypted on the user’s device and can only be decrypted by authorized collaborators, rendering it unreadable to any intermediate servers [18].
7. **You Retain Ultimate Ownership and Control** In this context, “ownership” refers to user agency and autonomy over their data. With local-first software, users have direct, unrestricted access to the files containing their data. They are free to back them up, move them, or manipulate them with other tools, without being limited by the APIs or terms of service of a cloud provider, restoring a level of control largely lost in the cloud era.

Achieving these ideals, especially that of seamless collaboration in a decentralized environment, requires foundational technologies capable of managing data replication and merging without a central coordinator.

3.4 Core Technology: CRDTs

CRDTs are a key enabling technology for the local-first paradigm. They are specialized data structures designed to be replicated across multiple devices, which can be updated concurrently and independently without requiring central coordination [8]. Their unique properties make them an ideal foundation for building collaborative, offline-first applications [1].

3.4.1 Principle of Strong Eventual Consistency

The core principle behind CRDTs is their ability to guarantee strong eventual consistency. This means that as long as all updates are delivered to all replicas, all replicas are mathematically guaranteed to converge to an identical state. This convergence is achieved automatically and deterministically through carefully designed merge functions that are commutative, associative, and idempotent, ensuring that concurrent updates can be combined in any order to produce the same final result without requiring complex consensus protocols. [8], [20]

3.4.2 Beyond Convergence: Causal Consistency

While Strong Eventual Consistency guarantees that all replicas will eventually agree on the state, it does not strictly dictate the order in which updates are applied or observed [20]. For collaborative applications, this is often insufficient. Users expect a system to respect cause-and-effect relationships; for instance, a reply to a comment should never appear before the comment itself [6, chap. 9].

Therefore, robust local-first systems typically aim for **Causal Consistency** [12], [20]. This model ensures that if operation A influenced operation B (i.e., A happened before B), then all processes must observe A before observing B . Operations that are concurrent (not causally related) can be observed in different orders [24]. By preserving these causal dependencies, the system maintains a logical flow of events that matches user expectations, even in the presence of complex network delays [6, chap. 9].

3.4.3 Role in Local-First Collaboration

CRDTs provide the underlying mechanism to realize the local-first ideal of seamless collaboration. They allow multiple users to independently edit a shared document on their local devices, even while offline [20]. When a network connection becomes available, these concurrent changes, represented as CRDT operations, are exchanged and merged, ensuring everyone's contributions are deterministically reconciled [8], [20].

3.5 Advanced Challenges in Local-First Architectures

Moving from the theoretical model of local-first software to a practical implementation for a complex application like e-commerce requires addressing significant challenges.

3.5.1 Semantic Conflicts and Invariant Preservation

A critical limitation of standard CRDTs is the distinction between **structural consistency** and **semantic consistency**. While CRDTs mathematically guarantee that all replicas will converge to the same state (structural consistency), they cannot inherently guarantee that this converged state satisfies application-specific business rules or invariants (semantic consistency) [9].

The Invariant Problem

This limitation is particularly acute in e-commerce scenarios involving finite resources, such as inventory management. Consider a product with a stock level of S . An invariant of the system is that stock must never be negative ($S \geq 0$).

Consider the following scenario where the initial stock is $S_0 = 1$:

1. **Concurrent Operations:** Two users, Alice and Bob, view the product while offline. Both observe $S = 1$.

- Alice purchases the item: $\text{op}_A = \text{decrement}(1)$. Her local state becomes $S_A = 0$.
 - Bob purchases the item: $\text{op}_B = \text{decrement}(1)$. His local state becomes $S_B = 0$.
2. **Merge and Violation:** When connectivity is restored, a standard Counter CRDT will merge these operations. The resulting state is calculated as:

$$S_{\text{merge}} = S_0 - 1 - 1 = -1 \quad (3.1)$$

3. **Result:** The system has achieved structural convergence (both nodes see -1), but it has violated the business invariant ($S \geq 0$), resulting in an oversold item. This constitutes a **semantic conflict**: the operations were valid in isolation but invalid when combined.

3.6 Technical Mechanisms of Persistence and UI

To address latency and availability challenges in distributed systems, software architectures rely on specific data management and user interface patterns. Understanding these mechanisms is necessary for evaluating how state is maintained and displayed across disconnected environments.

3.6.1 Mechanics of Optimistic UI and Speculative State

To simulate local agency within server-centric architectures, modern frameworks employ Optimistic UI patterns [1]. In the context of distributed systems, this is technically classified as a strategy for Latency Compensation. It decouples the feedback loop (interaction response) from the consistency loop (server confirmation) [1], [25].

In a standard request/response cycle, the client state S_{client} is a direct reflection of the last known server state S_{server} . In an optimistic model, the client immediately applies a mutation to its local view, transitioning to a tentative state $S_{\text{tentative}}$ before the network request is initiated:

$$S_{\text{tentative}} = S_{\text{known_server}} + \delta_{\text{pending}} \quad (3.2)$$

This creates a temporary “fork” in the state history. The application effectively maintains two parallel realities:

1. **The Truth:** The confirmed state from the server (S_{server}).
2. **The Prediction:** The optimistic state presented to the user ($S_{\text{tentative}}$).

Modern state management libraries manage this by maintaining a Mutation Queue. As the user performs actions, mutations are pushed to this queue and applied sequentially to the local cache. [25]

3.6.2 The Complexity of Divergence and Rollback

While this mitigates perceived latency, it introduces significant algorithmic complexity regarding failure recovery. If the server rejects the update (e.g., HTTP 409 Conflict or InventoryExhausted), the client cannot simply “undo” the last action if subsequent actions have occurred.

The system must perform a **Rebase**:

1. Revert $S_{\text{tentative}}$ back to the last stable state.
2. Discard the failed mutation.
3. Replay valid pending mutations against the clean state to see if they are still valid.

This requirement forces frontend developers to implement complex compensation logic. By doing so, they are effectively replicating distributed database recovery mechanisms within the UI layer, akin to implementing compensating transactions in a Saga pattern [26]. Furthermore, this duplication of logic requires maintaining parallel state machines on both the client and the server. As Moseley and Marks identify, the management of duplicated, mutable state is a primary driver of “accidental complexity” in software systems [27].

Consequently, while Optimistic UI is highly effective for commutative operations (like “toggling a like button”), it poses risks in retail environments where operations are non-commutative and order-dependent (e.g., claiming inventory). The literature suggests it serves as a “Band-Aid” solution that masks latency but does not solve the fundamental requirement for offline data integrity [1].

3.6.3 Command-Query Responsibility Segregation (CQRS)

As demonstrated by the complexities of Optimistic UI rollbacks, maintaining a single, unified data model for both reading and writing in a distributed, occasionally connected environment often leads to brittle state management and high “accidental complexity.” An architectural pattern frequently employed to mitigate this friction is Command Query Responsibility Segregation (CQRS).

Formalized extensively by Greg Young [28] and further popularized by Martin Fowler [29], CQRS challenges the traditional CRUD (Create, Read, Update, Delete) paradigm by strictly separating the conceptual models used to update information from the models used to read it. In a standard architecture, a single representation of state is used both to display data to the user and to enforce business invariants during mutations. CQRS splits this responsibility into two distinct operational pipelines: a “Write Model” that processes Commands, and a “Read Model” that answers Queries.

This explicit separation of intent (the Command) from the outcome (the projected state) aligns cleanly with the structural challenges of offline-capable e-commerce. It provides a formal framework acknowledging that while the local-first client has the agency to instantly

propose state changes offline, the server retains the ultimate authority to validate and apply them.

3.7 Regulatory and Domain Constraints in E-Commerce

Beyond the technical challenges of data replication and persistence, the architecture of e-commerce systems is fundamentally constrained by strict legal and security frameworks regarding payment processing.

Under the PSD2, online electronic payments initiated by the customer legally require SCA [30], [31]. This process, typically implemented via the 3D Secure protocol, requires the buyer to authenticate using two of three factors (Knowledge, Possession, Inherence). Because this involves a live, synchronous connection to the issuing bank’s Access Control Server at the exact moment of checkout, the initial authentication sequence cannot be executed offline. Furthermore, PSD2 mandates “dynamic linking,” requiring the authentication code to be uniquely tied to the specific transaction amount and payee in real-time [31, Article 97, par. 2].

Compounding these regulatory hurdles is the PCI-DSS. PCI-DSS strictly prohibits the local storage of **Sensitive Authentication Data** (SAD), such as **Card Verification Value** (CVV) codes, after authorization, and forbids the storage of raw **Primary Account Numbers** (PANs) on unprotected endpoint devices, such as local browser storage in a Progressive Web App [32, req. 3.3.1, 3.5.1]. While merchants utilize tokenization via payment gateways to bypass local data storage, generating an initial network token requires an active internet connection [33], [34]. Consequently, the current cryptographic and data protection paradigms governing electronic payments do not strictly prohibit offline checkouts, but they do structurally prevent synchronous, real-time authorization.

3.8 Theoretical Frameworks for System Evaluation

Evaluating a novel architectural paradigm like local-first software requires shifting from measuring absolute performance metrics to assessing relative trade-offs within constrained environments. Assessing whether an architecture can successfully reconcile client-side agency with the strict constraints of distributed data relies on formalizing two key dimensions: architectural viability and comparative system performance.

3.8.1 Defining Architectural Viability and Correctness

In distributed systems research, architectural viability evaluates whether a proposed design can theoretically satisfy its functional requirements without violating fundamental environmental or domain-specific constraints. It is not merely a measure of efficiency, but a strict threshold of constraint satisfaction.

Building upon the theoretical constraints established in previous sections, the evaluation of viability rests on two primary pillars:

- **Semantic Correctness under Partition:** Viability requires demonstrating that the architecture upholds critical business invariants (such as preventing negative inventory states, as detailed in Section 3.5) during the eventual resolution of partitioned states [8], [14]. As explored by Bailis et al. [3], a system is theoretically viable only if its chosen state management patterns can safely prevent or resolve these semantic conflicts without relying on synchronous global coordination for every operation.
- **Domain and Regulatory Alignment:** A viable theoretical model must maintain rigid modular boundaries that securely isolate competing consistency requirements [9], [35]. Specifically, it must successfully reconcile the optimistic availability of local-first data structures with the immutable, synchronous authorization mandates of financial regulations (e.g., PSD2 and PCI-DSS, detailed in Section 3.7).

3.8.2 The Theory of Comparative Benchmarking

Because environmental factors in mobile computing (bandwidth, latency, and hardware constraints) are highly variable, evaluating the performance of a distributed architecture in isolation yields metrics of limited scientific value. Literature on computer systems evaluation agrees that performance must be measured relatively, comparing the proposed system against an established baseline under identical, controlled workloads [36].

This necessitates the paradigm of comparative benchmarking. In the context of data-intensive distributed systems, foundational methodologies, such as those popularized by the **Yahoo! Cloud Serving Benchmark (YCSB)**, rely on isolating the architectural paradigm as the primary independent variable [37]. However, while frameworks like YCSB established this theory for server-side databases, evaluating a client-authoritative local-first architecture requires shifting the measurement boundary to the edge device [36]. Consequently, the benchmarking methodology must adapt these principles to the client side, replacing server-to-server traces with deterministic browser emulation to accurately quantify the performance delta between a legacy server-authoritative architecture and the novel local-first approach [36].

Methodology

The primary objective of this thesis is to investigate the architectural viability and performance implications of a “hybrid local-first” paradigm in e-commerce, reconciling the conflict between offline capabilities and strict inventory consistency.

To address this engineering challenge, this research adopts the DSR paradigm. As defined by Hevner et al. [10], DSR is a prescriptive paradigm aimed at generating knowledge through the construction of innovative artifacts. This approach is particularly suited to this study, as the goal is not merely to observe existing e-commerce systems, but to explore this architectural approach through a prototype instantiation, thereby generating transferable design knowledge.

This chapter details the methodological approach used to construct the primary artifact: a hybrid e-commerce application prototype. The methodology is structured to satisfy the three cycles of DSR as proposed by Hevner [38]: the **Relevance Cycle**, which grounds the research in the real-world problem of latency and network unreliability in mobile commerce; the **Rigor Cycle**, which draws upon the theoretical foundations of the CAP theorem, PACELC framework, and CRDTs and the **Design Cycle**, which iterates through the construction and evaluation of the artifact.

Finally, the chapter defines the rigorous evaluation framework used to benchmark the artifact’s utility. This includes a controlled laboratory simulation utilizing varying hardware and network profiles to compare the local-first system’s performance, resource utilization, and offline resilience against an industry-standard SSR-Hydrated SPA baseline.

4.1 Research Design: Design Science

Having established DSR as the foundational paradigm and outlined its governing cycles, this section operationalizes the methodology into a concrete plan of action. It details the practical execution of the research, transitioning from the theoretical justification to the structural steps required to systematically build and evaluate the hybrid local-first artifact.

4.1.1 The DSRM Process Model: A Roadmap

To ensure the resulting artifact generates transferable design knowledge, this research strictly follows the **Design Science Research Methodology (DSRM)** as described by Peffers et al. [11].

The four phases of the DSRM outlined below map directly to the structure of this chapter, as illustrated in Table 4.1.

DSRM Phase	Research Activity	Detailed In
1. Problem Identification	Analysis of the Consistency-Latency Trade-off	Section 4.2
2. Objective Definition	Derivation of the Decomposition Matrix	Section 4.3
3. Design & Development	Synthesis of the “Three-Pillar” Architecture	Section 4.4
4. Demonstration	Simulation of Relevance and Benchmarking	Section 4.5

Table 4.1: Mapping the DSRM to Chapter Structure

The final two phases of the DSRM, Evaluation and Communication, are addressed in subsequent chapters. The evaluation of the artifact is performed in the discussion chapter (Chapter 7), while the communication of findings and future outlook is synthesized across the discussion, outlook (Chapter 8) conclusion chapters (Chapter 9).

4.1.2 The Three Cycles of Inquiry

To ensure the academic validity of the artifact, the research design iterates through Hevner’s three cycles [38]:

- **The Relevance Cycle:** This cycle bridges the context of the environment (people, organizations, technology) with the design work. While a full field deployment is outside the scope of this thesis, the artifact is designed to address the verified need for robust software performance in environments with unstable or intermittent connectivity. The design requirements are derived from the constraints of modern mobile commerce, where network latency is often the primary bottleneck for user experience.
- **The Rigor Cycle:** This cycle connects the design activities with the knowledge base. The design is not based on ad-hoc decisions but is grounded in the theoretical constraints of the **CAP theorem** [19] and **PACELC** [7]. Furthermore, the implementation of the collaborative cart draws directly on the mathematical guarantees of **CRDTs** as defined by Shapiro et al. [8] and Almeida [20].
- **The Design Cycle:** The Design Cycle involves iterative refinement of the artifact. This research specifically explores the transition from pure CRDT models to a hybrid ‘Write-Path’ to address potential semantic conflicts like negative stock.

4.2 Domain Analysis: The Theoretical Framework

The methodology relies on a theoretical decomposition of the e-commerce application domain. A monolithic database model is rejected in favor of a granular analysis of data consistency requirements.

4.2.1 The Consistency Dilemma and PACELC

The fundamental theoretical lens for this thesis is the PACELC theorem. In the prototype, a **P**artition-**A**vailability/**E**lse-**L**atency (PA/EL) model is applied to the majority of the user experience. This decision is driven by the “Seven Ideals” of local-first software, specifically “No Spinners” and “The Network is Optional” [1]. By moving the data to the client, we eliminate the network from the read path, achieving the theoretical minimum for latency (L).

4.2.2 The Decomposition Matrix

However, a pure PA/EL system cannot safely handle financial transactions or rivalrous goods (inventory). Therefore, the methodology requires decomposing the application logic into distinct domains with varying consistency requirements.

To derive these specific domains, a functional analysis of standard e-commerce transaction boundaries was conducted. Data within the system was categorized along three dimensions: volatility (frequency of updates), user isolation (shared vs. individual state), and the necessity for global invariants (e.g., finite stock).

First, the Catalog represents high-volume, read-heavy data that changes infrequently and requires no user-specific isolation. Therefore, prioritizing availability and latency (PA/EL) over immediate global freshness is acceptable. Second, the Cart represents highly volatile, write-heavy data isolated to a specific user (or a small collaborative group). Since users expect to manipulate their cart seamlessly regardless of network conditions, it also demands PA/EL, but requires **S**trong **E**ventual **C**onsistency (SEC) to resolve concurrent offline edits. Finally, the Checkout/Inventory domain represents rivalrous goods where the business cannot tolerate semantic conflicts (e.g., overselling). This strict transaction boundary dictates a mandatory PC/EC model, where the system must block and wait for central authority. This analysis yields the following Decomposition Matrix (Table 4.2):

Domain	Data Characteristics	PACELC Fit	Consistency Requirement
Catalog	Read-heavy, non-rivalrous, high conflict tolerance	PA/EL (Availability over Freshness)	Eventual Consistency
Cart / Session	Write-heavy, collaborative, mathematically reconcilable	PA/EL (CRDT Convergence)	SEC
Checkout / Inventory	Transactional, rivalrous goods, zero conflict tolerance	PC/EC (Correctness over Latency)	Strict Serializability

Table 4.2: Decomposition Matrix of E-Commerce Consistency Requirements

4.2.3 Methodological Rationale for the Collaborative Cart

While standard e-commerce carts are typically isolated, single-user sessions or operate as simple persistent lists, this research deliberately elevates the shopping cart to a complex, multi-user collaborative workspace. This deviation from standard User Experience (UX) is derived from two methodological necessities:

1. **Simulation of Social Commerce Scenarios:** Emerging use cases, such as digitized B2B procurement and shared household shopping, provide a theoretical framework where state conflicts are highly probable [6], [13]. By modeling the cart as a shared state, the artifact can simulate “multi-device” concurrency (one user on phone and desktop) and “multi-user” concurrency (two distinct users) using the exact same synchronization logic.
2. **Structural Complexity for CRDT Benchmarking:** A standard flat list of products is insufficient to test the limits of CRDTs. Simple “Last-Writer-Wins” registers often suffice for flat lists. To rigorously evaluate the architectural viability of the selected CRDT implementation and the synchronization engine, the artifact introduces nested structural complexity via advanced features:
 - **Folders:** Introduce recursive tree structures, testing the system’s ability to handle move operations and parent-child reparenting without cycles.
 - **Notes:** Introduce rich-text sequence CRDTs, testing character-level conflict resolution.

- **Tags:** Introduce relational data modeling within a local-first context.

Consequently, the collaborative cart serves not just as a feature, but as the primary stress-test for the SEC requirement defined in the Decomposition Matrix.

4.2.4 Addressing Authoritative Gatekeeping

A critical methodological challenge identified in the domain analysis is authoritative gatekeeping. This occurs at the boundary where the user transitions from the **Eventual Consistency** domain (Cart) to the **Strict Consistency** domain (Checkout). The methodology must account for the validation of potentially stale or conflicting local states against the server’s authoritative state. This necessitates a “Forking History” [17] or rejection model, where the server acts as the final arbiter of valid state transitions.

4.3 Derivation of Design Requirements (Conceptual Framework)

To bridge the theoretical framework established in the Decomposition Matrix with a concrete software architecture, domain constraints must be translated into functional system capabilities. Accordingly, the four design requirements of the artifact are directly derived from these established Consistency Requirements:

1. To satisfy the PA/EL requirement of the Catalog domain, the system must establish a mechanism to distribute the read-state to the edge. This derives Requirement 1 (The Read-Path).
2. To satisfy the SEC requirement of the Cart/Session domain, the system must safely resolve concurrent offline mutations without relying on central locks. This derives Requirement 2 (The Collaborative-Path).
3. To enforce the strict PC/EC requirement of the Checkout domain, the local-first application must possess a fail-safe to bypass optimistic updates and consult the central database. This derives Requirement 3 (The Write-Path).
4. Finally, because the system relies on divergent states (local optimistic reads vs. authoritative server writes), a systemic mechanism is required to unify them and prevent state drift. This derives Requirement 4 (The Feedback Loop).

These four requirements serve as the foundational specifications for the artifact.

4.3.1 Requirement 1: The Read-Path (Offline-Available Catalog)

To satisfy the "Catalog" domain requirements (PA/EL), the system must implement a mechanism to maintain an Offline-Available Catalog. Unlike traditional network caching, which is ephemeral and relies on prior user navigation to populate, this requirement

mandates a proactively managed, persistent local datastore capable of serving catalog read operations entirely offline.

- *Justification:* The core premise of the local-first paradigm, specifically the “No Spinners” ideal [1], cannot be realized if user browsing is continuously gated by network latency. While replicating an entire enterprise e-commerce database is infeasible due to device storage quotas [39], a relevant, persistent subset of the catalog must reside on the client to ensure uninterrupted product discovery and navigation during network partitions.
- *Specification:* The architecture must invert the traditional client-server relationship for read operations. The client device must act as the primary, authoritative execution environment for catalog queries (e.g., searching, filtering, and loading product detail pages). The system must guarantee that the locally stored catalog is highly available and deeply queryable without requiring a synchronous network round-trip. By decoupling the core e-commerce browsing experience from connectivity, the application can achieve immediate interactivity and function reliably in intermittent or fully offline states.

4.3.2 Requirement 2: The Collaborative-Path (Mathematical Convergence)

To satisfy the "Cart" domain, the system requires a data structure capable of **mathematical convergence** to support the “Seamless Collaboration” ideal.

- *Justification:* Traditional conflict resolution (e.g., “Last Writer Wins”) is insufficient for shopping carts. If User A adds item X and User B adds item Y concurrently, the result should contain both X and Y.
- *Specification:* The system must utilize CRDTs. Specifically, delta-state CRDTs are preferred to minimize bandwidth during synchronization and for their resilience in high-latency networks. The artifact must support multi-user concurrent editing of the cart state without locking, ensuring that all replicas eventually reach the same state [20].

4.3.3 Requirement 3: The Write-Path (Synchronous Authority)

To satisfy the "Checkout" domain, the system requires a **Synchronous Authority**.

- *Justification:* CRDTs ensure structural consistency (all nodes agree on the state) but cannot inherently guarantee semantic consistency (e.g., “Stock ≥ 0 ”).
- *Specification:* Critical mutations (Checkout, Payment) must bypass the local replication stream and be sent directly to the server via a synchronous channel, such

as **R**emote **P**rocedure **C**all (RPC) or **R**Epresentational **S**tate **T**ransfer (REST). This allows the server to enforce global invariants through **A**tomicity, **C**onsistency, **I**solation, **D**urability (ACID) transactions and reject invalid states before they are committed [17].

4.3.4 Requirement 4: The Feedback Loop (State Reconciliation)

To maintain a coherent user experience, the system requires **State Reconciliation**.

- *Justification:* When a synchronous write (Req 3) changes server state, this change must be propagated back to the client’s local replica (Req 1).
- *Specification:* The architecture must implement a CQRS style loop. The client issues a Command (RPC); the server processes it; the change is written to the primary Database (DB) the **C**hange **D**ata **C**apture (CDC) system detects the write and pushes the update down the Replication Stream to the client.

4.4 Artifact Instantiation

To validate the design requirements, a functional prototype was constructed. The selection process prioritized tools that natively support the “Local-First” paradigm while offering the necessary primitives for hybrid consistency.

4.4.1 Comparative Analysis and Selection Rationale

To ensure the robustness of the artifact, a comparative analysis was conducted to select the most appropriate tools from the emerging local-first ecosystem.

Synchronization Engine: ElectricSQL vs. RxDB

The strongest alternative considered was RxDB [40], a popular local-first JavaScript database. RxDB is widely recognized for its reactive capabilities and extensive plugin system.

However, ElectricSQL [41], a local-first synchronization engine that facilitates bidirectional database replication between a centralized PostgreSQL database and a local database, was selected over RxDB for two critical reasons. First, RxDB operates on an “Open Core” commercial model where significant performance features, specifically optimized storage adapters and advanced persistence layers, are locked behind a proprietary paywall. Reliance on proprietary modules restricts the artifact’s reproducibility, violating the open-science ethos of the DSR framework.

Second, RxDB functions primarily as a **N**ot **O**nly **S**QL (NoSQL) document store. Adopting it would introduce a “Relational Impedance Mismatch,” requiring the complex e-commerce schema (which relies on strict foreign keys between Products, Variants, and Stock) to be denormalized into documents. ElectricSQL mitigates this by synchronizing directly with PostgreSQL [42]. Functioning as a standard object-relational database system, PostgreSQL preserves the relational integrity of the data model throughout the stack. This

is essential for satisfying Requirement 3 (Synchronous Authority).

CRDT Implementation: Yjs vs. Automerge

The two dominant libraries for implementing CRDTs in JavaScript are Automerge [43] and Yjs [44]. While Automerge utilizes a purely functional approach, empirical benchmarks indicate that Yjs provides measurable advantages in resource efficiency and processing speed. Specifically, Yjs demonstrates a notably smaller bundle size and faster execution and parsing times when processing large-scale, real-world editing traces, whereas Automerge exhibits prohibitive memory constraints under continuous, high-volume operation [45].

Yjs was selected for this prototype due to its highly efficient encoding speeds and superior computational performance in text and linear data manipulation, as evidenced by empirical benchmarks. It functions as a CRDT framework designed to manage state distribution and resolve conflicts in collaborative applications. Furthermore, its ecosystem provides established bindings for the chosen framework (y-electric), facilitating the requirement for seamless collaboration without necessitating significant custom engineering.

RPC Layer: tRPC vs. GraphQL vs. REST

For the synchronous “Write-Path” (Req 3), the architectural requirement is merely a reliable synchronous transport layer. Theoretically, standard implementations of REST, gRPC [46], or GraphQL [47] are all functionally capable of transmitting the necessary transaction payloads.

However, TypeScript **R**emote **P**rocedure **C**all (tRPC) [48] was selected over these alternatives to enforce End-to-End Type Safety without the risk of “Interface Drift”, a common source of bugs in distributed systems where API documentation deviates from the actual implementation. It operates as an API communication layer that leverages TypeScript inference to enforce strict type safety between the client and server without requiring explicit schema generation.

While GraphQL offers strong typing, it introduces significant architectural overhead through the management of **S**chema **D**efinition **L**anguage (SDL) and resolvers, which was deemed disproportionate for the limited scope of transaction mutations. Similarly, standard REST APIs often lack strict compile-time guarantees for payload structures.

In contrast, tRPC provides a rigorous mechanism for schema inference. By integrating directly with the rest of the technology stack, such as the Drizzle **O**bject-**R**elational **M**apping (ORM) [49] (a TypeScript Object-Relational Mapper) and the TanStack Start [50] (a full-stack React application framework), tRPC allows the system to infer TypeScript types directly from the server-side database definition. This creates a single source of truth for data structures, ensuring that the “Contract” between the local optimistic state and the authoritative server is validated at compile-time. This reduces the surface area for runtime serialization errors during critical financial transactions.

While the State of Research acknowledges the theoretical robustness of academic frameworks such as ConLoc [9] and LoRe [12], these approaches impose a substantial ‘specification burden’ via SMT solvers and specialized compilers. Such overhead is often prohibitive for rapid commercial development cycles. Consequently, this research diverges from formal compile-time verification to pursue a strategy of Runtime Enforcement via CQRS. Drawing upon the CQRS pattern described in Section 3.6.3, the artifact operationalizes hybrid consistency models, specifically PA/EL for browsing and PC/EC for checkout, through emerging industry standards rather than bespoke languages. By leveraging ElectricSQL

for relational replication and tRPC for transactional authority, this methodology demonstrates that rigorous consistency can be achieved pragmatically within a standard software engineering context.

4.4.2 Architecture Synthesis

The resulting “Three-Pillar Model” operates through three distinct phases:

1. **Optimistic UI (Phase 1):** The user performs an action. The UI updates immediately using the local database (PA/EL).
2. **Dual Dispatch (Phase 2):**
 - Non-critical data (Cart item quantity) is written to the Local DB and synced via the replication stream or CRDT.
 - Critical data (Checkout) is sent via tRPC to the server (PC/EC).
3. **Reconciliation (Phase 3):** The server writes to Postgres. ElectricSQL detects the CDC event and pushes the confirmed state back to the client, overwriting the optimistic state if necessary.

4.5 Evaluation Methodology

The final phase of the DSR framework requires a rigorous demonstration of the artifact’s utility. This study defines a comprehensive evaluation approach to evaluate the proposed “Three-Pillar” hybrid architecture, comprising quantitative performance benchmarking against an industry-standard baseline and deterministic functional correctness testing.

Rather than observing a production environment with uncontrollable variables, the methodology employs a “Laboratory Simulation of Relevance” using the Playwright testing framework [51]. Playwright is an open-source automation library, designed for deterministic end-to-end testing across modern web browser engines.

Playwright was selected as the automation backbone for this evaluation due to its native architecture, which operates directly over the **Chrome DevTools Protocol (CDP)** [52]. Unlike older frameworks that rely on intermediary drivers, Playwright allows for low-level, deterministic control over browser engines (Chromium, Firefox, and WebKit). Crucially for this study, its native APIs enable precise network interception to simulate offline states, varying latency, and bandwidth throttling. It also provides isolated ‘Browser Contexts’. [53] These contexts allow for the creation of completely independent, concurrent user sessions within a single test execution, which is required to evaluate the collaborative conflict-resolution mechanics of the artifact.

To ensure ecological validity and holistically represent the varied landscape of real-world mobile commerce, the simulation environment is partitioned into four distinct profiles.

To account for variances between host machines and prevent overly powerful hardware (e.g., AMD Ryzen 7 7800X3D) from skewing results, the testing suite includes a pre-test dynamic calibration step. This step applies hardware throttling multipliers against the host **Central Processing Unit (CPU)** alongside standardized network presets:

Hardware Constraints (CPU)

- **Throttled CPU (3.6x Slowdown):** Calibrated directly to match the Chrome DevTools “mid-tier mobile” preset.
- **High-end CPU (1.2x Slowdown):** Calculated as one-third of the mid-tier penalty. This establishes a realistic baseline for modern flagship mobile devices without allowing desktop-grade processors to artificially inflate performance.

Network Constraints

- **Fast 4G:** 9 Mbps uplink and downlink speeds, 165 ms baseline Round Trip Time (RTT).
- **Slow 3G:** 400 Kbps uplink and downlink speeds, 2000 ms baseline RTT.

Combining these hardware and network constraints yields the following four evaluation profiles:

- **Baseline:** High-end CPU paired with a Fast 4G network.
- **Budget:** Throttled CPU paired with a Fast 4G network.
- **Commuter:** High-end CPU paired with a Slow 3G network.
- **Worst-Case:** Throttled CPU paired with a Slow 3G network.

This partitioning addresses the unique performance challenges of mobile environments, which are heavily constrained by reduced processing power, lower network bandwidth, and higher latency compared to desktop computers [5]. By combining these extremes, the profiles capture genuine user scenarios ranging from flagship device owners experiencing intermittent connectivity during a commute to users navigating the web on mid-to-low-tier hardware. The network presets are based on Chrome DevTools’ “Fast 4G” and “Slow 3G” profiles [54], which are widely accepted industry standards that utilize artificial throttling, a recognized methodology to mimic real-world delays and ensure simulated performance more closely reflects the Quality of Service observed by actual end-users [36].

4.5.1 The SSR-Hydrated SPA Control Group

To move beyond theoretical assertions of performance, the local-first software artifact is benchmarked against a functionally equivalent SSR-Hydrated SPA control group.

The comparison application was purpose-built for this evaluation based on the existing local-first prototype. To isolate the architectural paradigm as the sole independent variable, the existing application framework (TanStack Start) was re-configured to operate as a modern, industry-standard SSR-Hydrated SPA [22]. The local-first specific components (ElectricSQL, TanStack DB, and Yjs) were stripped and replaced with state-of-the-art server-side rendering paradigms. This baseline specifically relies on smart code-splitting, **HyperText Markup Language** (HTML) hydration, and server-side querying [22] and caching via TanStack Query [55] and TanStack Start server functions. TanStack Query acts as the primary asynchronous state management and data fetching library for this baseline, ensuring that the control group represents a optimized, rather than a naive, server-

authoritative implementation. Crucially, to ensure a highly optimized and realistic control environment, the SSR-SPA implements aggressive server-side pagination. The initial HTML payload and subsequent hydration are strictly limited to the first 24 products. Furthermore, the application employs virtualized infinite scrolling for subsequent data fetches, meaning DOM nodes are dynamically recycled. This architectural decision deliberately prevents DOM bloat and standardizes the client-side rendering tax regardless of the underlying catalog size.

To guarantee environmental validity and prevent hosting disparities from skewing the results, the exact same physical server infrastructure was utilized to host both the Local-First sync engine and the resulting SSR build. To serve the application, the methodology employs Vite [56] as the frontend build tool and places it behind a Caddy [57] reverse proxy to route web traffic.

4.5.2 Statistical Approach and Dataset Scaling

To evaluate scalability, both architectural paradigms are subjected to three distinct dataset sizes. These benchmarks are directly informed by the 2026 State of Shopify report to ensure the evaluation reflects real-world e-commerce catalog distributions [58]:

1. **Boutique Store (100 items):** The Shopify report indicates that over 68% of active stores sell fewer than 100 products, with another 11.2% (roughly 314,944 stores) falling in the 100-249 range. The 100-SKU baseline therefore represents the catalog size of the vast majority of typical small-to-medium merchants on the web today.
2. **Mid-Sized Store (5,000 items):** Serving as an intermediary benchmark, this size represents established, high-growth merchants. The report identifies over 114,000 stores in the 1,000-4,999 product range, making 5,000 items a rigorous upper-bound stress test for standard mid-market catalogs before entering the enterprise tier.
3. **Large Retailer (25,000 items):** Representing the upper echelon of e-commerce platforms, this dataset mimics the top 0.3% of stores (the 25,000-99,999 product bucket, comprising nearly 10,000 live stores, often running enterprise solutions like Shopify Plus). This size is specifically designed to test the extreme scalability limits, browser memory constraints, and initial payload thresholds of both the local-first sync engine and the SSR hydration processes.

For the purpose of these evaluations, a single product record consists of basic text metadata, one baseline pricing tier, and a single inventory ledger entry. To strictly isolate the performance of the synchronization engine, these simulated payloads deliberately omit heavy assets like product images or complex variant hierarchies.

To mitigate the statistical skew caused by transient system anomalies, browser garbage collection, and outliers inherent in automated profiling, all quantitative temporal measurements adopt a strict distribution-based approach. Every quantitative test scenario and metric recording is iteratively repeated 30 times for each of the 24 configurations (Architecture $\times$ Profile $\times$ Dataset). As outlined by Montgomery and Runger, 30 iterations provide a sufficiently large sample size for the sample mean to approximate a normal distribution, thereby ensuring statistical stability in the metric aggregation [59, chap. 7].

Crucially, because the state of the browser’s IndexedDB and cache introduces significant variability when benchmarking local-first software, each test execution occurs within a

strictly isolated, clean-slate browser context. Performance outcomes are reported using the median, the average (mean), and the 95th percentile (P95), ensuring a comprehensive representation of both the typical user experience and realistic tail latencies. As articulated in “The Tail at Scale” [60], relying solely on median or mean latency in web performance fails to reflect the actual user experience for a large portion of traffic. Reporting the 95th percentile is necessary to capture and optimize for the high-latency anomalies that ultimately dictate overall system responsiveness and user satisfaction.

4.5.3 Performance and Resource Utilization

This dimension tests the computational and temporal costs imposed on the client device, directly comparing the Local-First approach against the SSR-Hydrated SPA baseline.

Time to Interactive & Initial Synchronization

This test measures the duration from the initial page request to a fully interactive state. For the local-first artifact, this includes the penalty of the initial database sync; for the SSR-Hydrated SPA, it measures the time to first byte (**T**ime to **F**irst **B**yte (TTFB)), asset download, and hydration.

- *Rationale:* Market research indicates typical mid-market e-commerce catalogs range between 1,000 and 10,000 **S**tock **K**eeping **U**nits (SKUs) [58]. Benchmarking across the three dataset sizes verifies if the initial Local-First data payload remains viable across the four network/device profiles before degrading the user experience compared to a standard SSR-Hydrated SPA load.

Interaction Latency (Read Operations)

This test records the time required to execute complex read operations (search/filter).

- *Rationale:* To prove the PA/EL (Availability/Low Latency) benefit of the hybrid architecture, local execution times must demonstrably and consistently outperform the network round-trips of the SSR-Hydrated SPA baseline, particularly in the “Commuter” and “Worst-Case” profiles.

Storage Footprint and Client Resource Tax

This involves tracking the cumulative size of the browser’s IndexedDB, alongside JavaScript Heap Memory snapshots and CPU usage during the synchronization process.

- *Rationale:* High memory consumption or main-thread blocking would negate the UX benefits of a local database. Measuring these metrics against the SSR-Hydrated SPA baseline ensures the architecture is realistically deployable on structurally constrained mobile devices (the “Budget” profile).

4.5.4 Network Efficiency

These tests verify the bandwidth reduction capabilities and the “No Spinners” ideal of the architecture.

Bandwidth Consumption (Navigation Flow)

Total data transfer (in KB) is tracked during a standardized multi-page user journey (Homepage -> Search -> Product Page).

- *Rationale:* This validates the hypothesis that while the local-first model incurs a high initial bandwidth cost, it approaches the theoretical minimum of data transfer for subsequent route navigations, fundamentally outperforming the continuous data fetching and background cache revalidation required by the SSR-Hydrated SPA paradigm.

Delta Update Payload (Local-First Only)

This test modifies a single item’s property on the server and measures the raw byte size of the granular delta payload sent to the client via ElectricSQL.

- *Rationale:* Re-fetching entire resources is antithetical to offline-first design. This confirms the background replication stream is efficient enough to maintain eventual consistency without congesting the network.

4.5.5 Distributed State Consistency and Conflict Resolution

The correctness of the local-first system is evaluated by simulating edge cases that typically cause invariant violations in distributed, partitioned networks [9], [17].

Conflict Scenario A: Concurrent Inventory Mutation (Split-Brain)

The test simulates two clients competing for a single inventory item while one is offline, ultimately attempting to process a checkout upon reconnection.

- *Rationale:* This is the critical stress test of the hybrid architecture. It proves that the local CRDT can safely accept optimistic offline additions (Availability), while the server deterministically handles the conflict upon reconnection via the synchronous tRPC write-path to prevent negative stock (Consistency).

Conflict Scenario B: Structural Concurrency

The test utilizes Playwright’s isolated Browser Contexts to simulate two independent, concurrent sessions (“User A” and “User B”). The scenario begins with a collaborative cart containing a single folder that holds one item. Both contexts are deliberately taken offline via Playwright’s network routing features before they independently modify the nested structural tree. Specifically, User A adds a new item to the folder, while User B concurrently deletes that folder. The network connection for both contexts is then restored to observe the reconciliation process.

- *Rationale:* This explicitly evaluates Requirement 2 (Mathematical Convergence). While flat lists can rely on simple “Last-Writer-Wins” logic, nested cart structures risk generating cycles or orphaned data during concurrent structural mutations

[6, chap. 5]. To validate architectural claims regarding structural complexity and robust offline collaboration, the test evaluates three deterministic assertions upon network restoration:

1. Stability: Neither browser context throws a JavaScript runtime exception.
2. State Parity: Both Context A and Context B resolve to an identical structural state.
3. Resolution Logic: The conflict resolution logic is strictly verifiable.

Time to Consistency (TTC)

This metric isolates the duration elapsed from the moment a client regains connectivity to the successful processing and UI rendering of a server-side state update. The stop-watch trigger is the exact moment the **Document Object Model** (DOM) updates to reflect a modified watched product.

- *Rationale:* A core challenge of eventual consistency is the “staleness” window. A sub-second TTC validates that the reconciliation loop functions fast enough to prevent user confusion.

Server-Authoritative Gatekeeping

A client-side mutation is simulated to bypass local business logic (e.g., maliciously altering a product price in the client state).

- *Rationale:* Validates Requirement 3 (Synchronous Authority). The system must maintain a “Trust Boundary,” strictly rejecting invalid states at the server level regardless of the local optimistic view.

4.5.6 Offline Feature Availability

The network interface is deliberately disabled. A Pass/Fail audit on core functionalities is conducted.

- *Rationale:* Confirms the strict isolation of the read-path and the write-path. Browsing and cart management must remain fully operational (PA/EL), while authoritative checkouts are securely blocked (PC/EC).

4.6 Limitations and Scope

This research design has several inherent limitations.

Network Simulation vs. Physical Reality

The evaluation relies on Playwright and Chrome DevTools to simulate latency and partitions. While standard in web performance research, this does not perfectly emulate

the stochastic nature of physical packet loss, jitter, or “lie-fi” (connected but unresponsive) states found in real-world cellular networks [1]. Therefore, the results represent a controlled, best-case scenario for the protocol’s resilience.

Client-Side Resource Boundedness

This methodology assumes the client device possesses sufficient computational power and memory to index and query the local database. Performance on low-end hardware (represented by the Budget/Worst-Case profiles) may highlight the degradation curve of local-first compared to the SSR-Hydrated SPA approach, which offloads computation to the server.

Prototype Nature & Feature Parity

The artifact constructed is a functionally scoped prototype focused on core commerce primitives. It excludes resource-intensive features standard in production environments, such as third-party analytics trackers and personalized recommendation engines. The performance metrics observed represent an idealized scenario; auxiliary subsystems would introduce overhead that could diminish the raw latency gains observed in this isolated study.

Dataset Scaling

Exploratory testing revealed that catalog sizes approaching 50,000 items resulted in unfeasible initial load times and browser memory saturation due to the upfront payload requirements of the local-first sync engine. Therefore, 25,000 items was established as the maximum viable benchmark for this architecture in its current state.

Results

As dictated by the DSRM, the primary outcome of the design cycle is the instantiated software artifact. Therefore, this chapter presents the developed “Three-Pillar” hybrid architecture as the first major result of this thesis. To address the first half of the primary research question, “the extent to which a hybrid architecture is required to reconcile offline capability with data consistency”, this chapter details the technical realization of the artifact, known as **Partslist**.

The text explores the system’s CQRS-driven state management strategy, detailing the integration of ElectricSQL and TanStack DB to construct the optimistic, highly available read layer for catalog management. Furthermore, it breaks down the implementation of Yjs for managing the complex, collaborative state of the shopping cart, alongside the utilization of tRPC for the synchronous, server-authoritative write path that governs checkout transactions. Finally, it examines the reactive feedback loop responsible for reconciling local optimistic updates with the server’s authoritative truth.

5.1 The “Three-Pillar” Hybrid Architecture

The **Partslist** artifact bridges the theoretical gap between local-first responsiveness and server-side invariant enforcement. The implementation does not rely on a monolithic consistency model. Instead, it operationalizes the decomposition matrix derived from the PACELC framework, assigning distinct consistency strategies to different functional domains of the application.

The system is constructed using a CQRS state management strategy (see Figure 5.1). This architecture bifurcates the data flow into a local-first read path (PA/EL) for browsing and collaboration, and an authoritative write path (PC/EC) for financial transactions.

5.1.1 Architectural Components and Data Flow

The system consists of three primary nodes: the Client (Browser), the Sync Engine (ElectricSQL), and the Authoritative Server (PostgreSQL + tRPC) which interact in a continuous, three-phase data flow cycle as detailed in Figure 5.1:

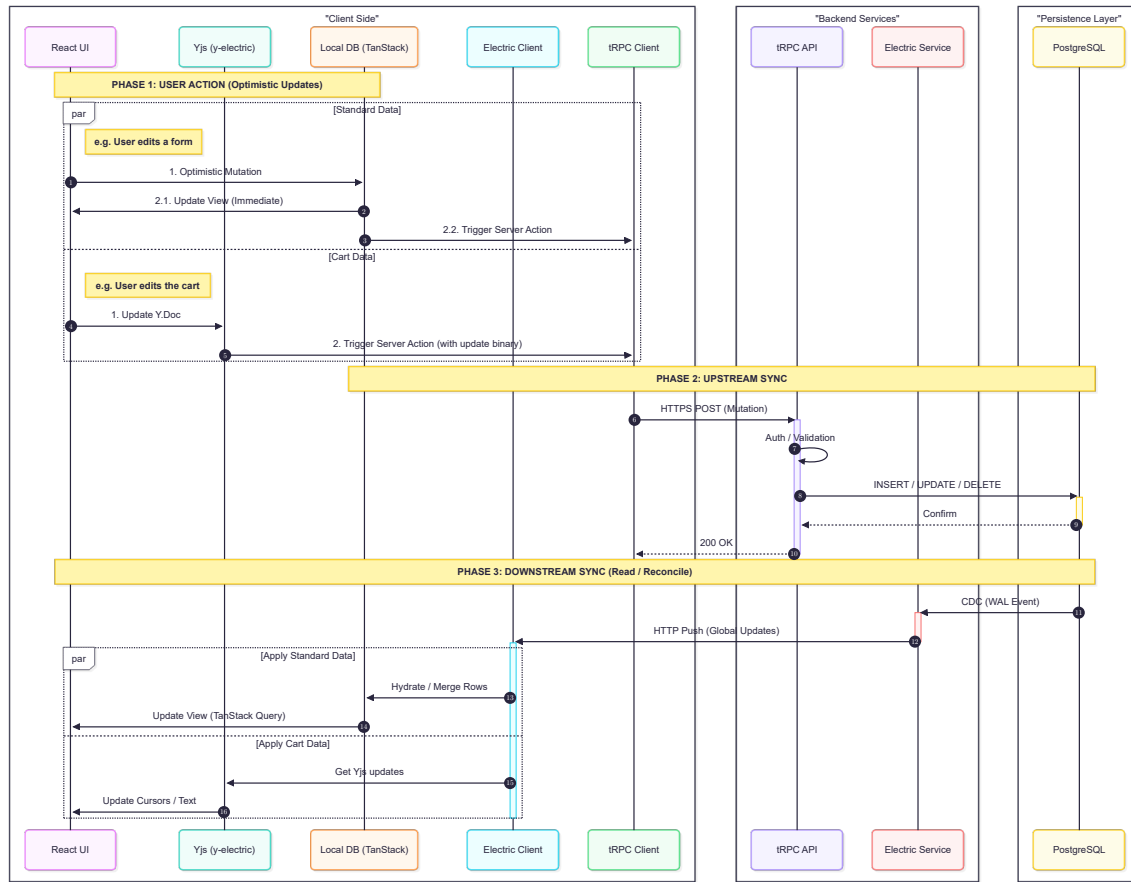


Figure 5.1: Architecture Diagram showing the data flow between Client, Sync Engine, and Authoritative Server

Phase 1: Optimistic Local Execution (The Read Path & Collaborative State)

To minimize interface latency, the client acts as the primary source of truth for the UI. When a user interacts with standard application data, mutations are instantly applied to TanStack DB. Similarly, collaborative actions (like shopping cart modifications) are immediately applied to a local Yjs CRDT document. Both paths update the view instantly without network overhead, satisfying the SEC requirement for real-time co-editing.

Phase 2: Upstream Synchronization (The Authoritative Write Path)

Concurrently, the client syncs local modifications to the backend. To enforce global invariants (e.g., inventory limits) and handle shared data, both the collaborative state and the authoritative state are posted using the exact same tRPC channel. For authoritative operations, this synchronous channel executes and validates transactions against the central PostgreSQL database, ensuring strict serializability for critical operations like checkout.

Phase 3: Downstream Synchronization and Reconciliation

Once the authoritative transaction is committed to PostgreSQL, a CDC protocol captures the event. The ElectricSQL Sync Engine actively pushes these verified updates back down the replication stream to all connected clients. The local TanStack DB and Yjs instances ingest this stream, seamlessly reconciling and replacing the temporary optimistic state with the globally authoritative truth.

5.2 Pillar I: The Optimistic Read Layer (Catalog Management)

The implementation of the catalog management system addresses the “Subset Problem” inherent in partial replication: how to sync only relevant data to a mobile device while maintaining referential integrity.

5.2.1 Shape-Based Replication

To prevent the client from downloading the entire database (which would violate storage constraints on mobile devices), the implementation utilizes the concept of “Shapes.” A Shape is a subscription to a specific subset of the relational graph.

Critically, the definition of these Shapes is performed exclusively on the server-side, creating a security boundary that is immune to client-side injection attacks. In the artifact, this is implemented via a proxy API route (e.g., `/api/categories`). The client cannot request arbitrary data ranges; instead, the server autonomously constructs the Shape definition using the authenticated session context.

```
// Source: API Route ensuring authorized data subsets
const originUrl = prepareElectricUrl(request.url);
originUrl.searchParams.set("table", "orders");
const filter = `user_id = '${session.user.id}'`;
originUrl.searchParams.set("where", filter);
return proxyElectricRequest(originUrl);
```

This ensures that the client holds a coherent graph of data (e.g., Categories, Products, and Assets) that is carved out of the central database but maintains all internal foreign key relationships.

Caveat on Replication Scope

While the shape-based architecture supports granular data fetching, the application ultimately pre-loads *all* defined shape data upon mount to guarantee full local-first, offline availability. True partial replication was deemed unnecessary for this prototype. The catalog acts as the largest driver of storage capacity and must be fully replicated regardless to support seamless searching, even while online. Because the remaining datasets do not contribute significantly to overall storage constraints, a complete sync was favored over a partial one. Potential solutions, such as predictive partial replication, are explored in the outlook (Chapter 8).

5.2.2 Reactive Query Engine

Once data is replicated to the client’s local storage, it is surfaced to the UI via TanStack DB. The implementation replaces traditional asynchronous `fetch` calls with live queries, i.e., queries that automatically update the UI when the underlying data changes.

A key technical achievement here is the implementation of cross-collection joins on the client. As demonstrated in the source code, the system can join `products` with `categories` dynamically:

```
// Source: Client-side join without network latency
const { data: products } = useLiveQuery((q) => q
  .from({ product: productsCollection })
  .join({ category: categoriesCollection }, ({ category, product }) =>
    eq(category.id, product.category_id)
  )
);
```

This implementation enables synchronous sorting and filtering. Because the data resides locally, the **Time to Interactive (TTI)** for these operations is minimized, satisfying the latency constraints defined in the methodology.

The practical result of this localized query execution is a highly responsive search interface, allowing users to filter the catalog without network-induced loading states (see Figure 5.2).

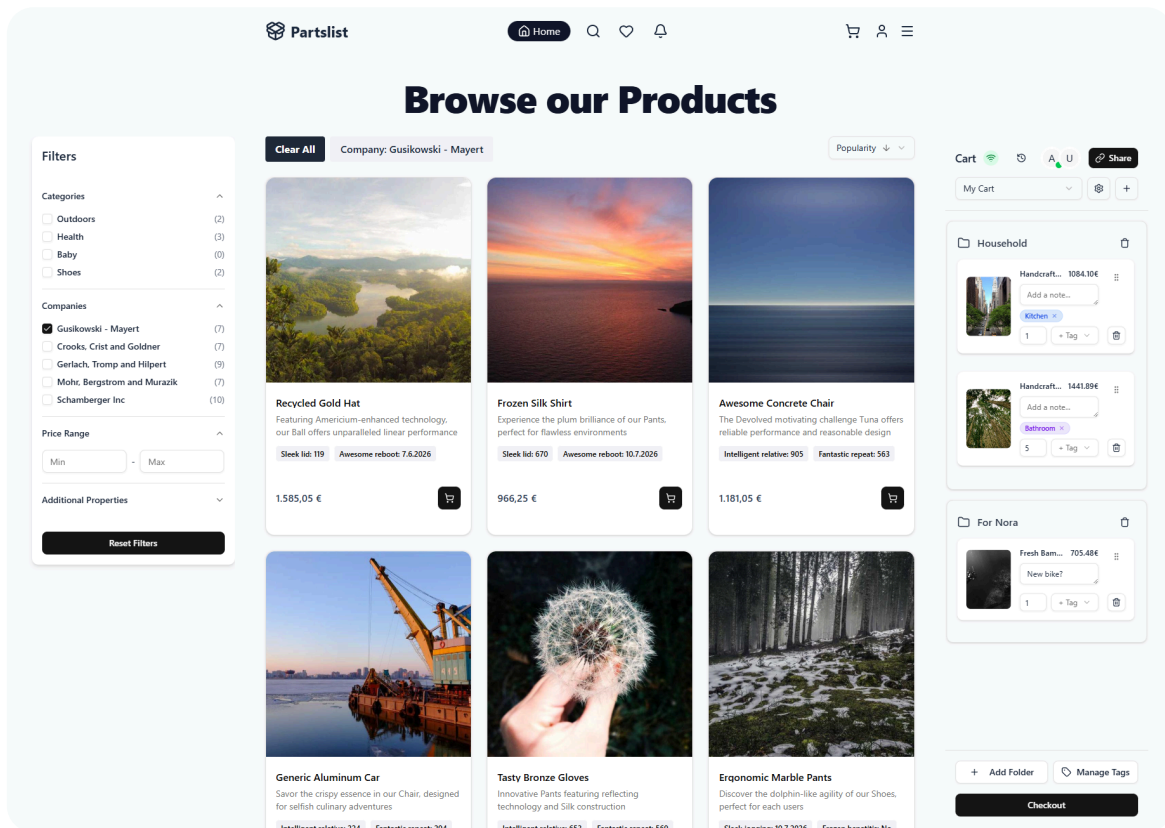


Figure 5.2: The Partslist search page interface, demonstrating instantaneous local filtering and sorting driven by the reactive query engine.

5.2.3 Offline Application Shell and Asset Caching

While the reactive query engine ensures that data is available locally, a true local-first architecture requires the application bundle itself (HTML, JavaScript, Cascading Style

Sheets (CSS), and static assets) to be available without network access. Without this, an offline user would be unable to boot the application to access their locally replicated data.

To solve this, the implementation utilizes a Service Worker powered by `servist`, which acts as a programmable network proxy within the browser.

The caching strategy is implemented in two distinct phases:

1. **Build-Time Asset Injection:** During the build process, `@servist/build` is utilized to analyze the output directory and generate a manifest of all static assets. This manifest is injected directly into the Service Worker (`injectManifest`). Upon installation, the Service Worker precaches the entire SPA shell. For navigation requests, a fallback (`/_shell.html`) is strictly targeted, ensuring the application can mount and execute routing logic entirely offline.
2. **Runtime Authentication Caching:** A significant challenge in offline-capable applications is handling authentication states when the identity provider is unreachable. To address this, the Service Worker implements a specialized `NetworkFirst` runtime caching strategy for the authorization session endpoint (`/api/auth/get-session`).

```
// Source: Service Worker runtime caching for offline authentication
servist.registerRuntimeCaching({
  matcher: ({ url }) => url.pathname === "/api/auth/get-session",
  handler: new NetworkFirst({
    cacheName: "auth-session-cache",
    networkTimeoutSeconds: 5, // Fallback to cache if network is slow/offline
    matchOptions: { ignoreVary: true, ignoreSearch: true }
  })
});
```

By caching the session, the application allows offline users to bypass the login and boot into the application using their last known verified identity. Once the frontend mounts, TanStack DB and ElectricSQL handle the authorization of the local data reads, ensuring a seamless and secure offline user experience.

Furthermore, to guarantee uninterrupted usage, the application is configured to eagerly preload all JavaScript route chunks and all database data immediately upon the initial mount. This proactive fetching strategy ensures complete offline functionality across the entirety of the application footprint. By doing so, a user can navigate to any section and access all required data without a network connection, even if a specific route had not been previously visited during a prior online session. The technical implementation of how this eager preloading is used to completely eliminate render flicker is detailed in [Appendix B-E](#).

5.3 Pillar II: The Collaborative State (The Shopping Cart)

The shopping cart serves as the primary testbed for the collaboration requirement. The implementation moves beyond simple “Last-Writer-Wins” registers, utilizing Yjs to manage

complex nested states. For details on how distinct collaborative sessions are isolated from one another at the React component level to prevent data leaks, refer to Appendix B-C.

5.3.1 The CRDT Data Model

The cart is not modeled as a simple list but as a `Y.Doc` containing specific CRDT structures:

- **nodes (`Y.Map`):** A flat map containing all cart items and folders. Crucially, the hierarchy is maintained via `parent_id` references rather than a nested **JavaScript Object Notation** (JSON) tree. This “flattened hierarchy” simplifies the move logic and prevents the complexity of recursive CRDT merges. To see how drag-and-drop reordering is implemented within this flat structure without index conflicts, see Appendix B-B.
- **snapshots (`Y.Array`):** This structure stores history states, enabling undo/redo functionality specifically for the cart context.

5.3.2 Network Adapter Configuration and Transport Consolidation

A distinguishing feature of Yjs is its modular network architecture, which allows the developer to configure the connection provider independently of the data structure. While standard implementations often rely on **Web Real-Time Communication** (WebRTC) or generic WebSocket providers (e.g., `y-websocket`), this architecture routes upstream Yjs updates through the exact same tRPC channel used for the authoritative state.

By consolidating the upstream sync for both the collaborative state and authoritative transactions over a single tRPC channel, the system avoids the overhead of maintaining a secondary WebSocket connection for ephemeral state. Furthermore, this approach prevents the duplication of authentication middleware; the security context established for the tRPC endpoints automatically extends to the collaborative layer. Consequently, the central PostgreSQL database acts as the authoritative state and the relay mechanism for all collaborative events, unifying the upstream communication strategy across both Pillars I and II. To understand how real-time user presence and awareness are broadcasted over this same stream, see Appendix B-D.

5.4 Pillar III: The Authoritative Gate (Checkout & Inventory)

While the catalog and cart prioritize availability (PA/EL), the checkout process enforces strict consistency (PC/EC) to satisfy both inventory invariants and the synchronous SCA requirements of PSD2 (as outlined in Section 3.7). This is implemented via an upstream sync pattern where both the collaborative state and the authoritative state use the same tRPC channel for syncing and posting, while clearly delineating the “Optimistic” from the “Authoritative” processing on the server side.

5.4.1 The Synchronous Write Path (tRPC)

Unlike cart interactions, the checkout action does not write to the local database. Instead, it triggers a tRPC mutation. This choice was driven by the benefit of end-to-end type safety. The server and client share the exact same Zod validation schemas.

```
// Source: tRPC Mutation enforcing consistency
export const categoriesRouter = router({
  create: authenticatedProcedure
    .input(createCategorySchema) // Shared Zod Schema
    .mutation(async ({ ctx, input }) => {
      // Server-side transaction
      const result = await ctx.db.transaction(async (tx) => { ... });
      return result;
    })
});
```

This architecture ensures that the “contract” for a valid transaction is checked at compile time. At runtime, the server performs the authoritative check within a PostgreSQL transaction.

5.4.2 Snapshotting for Historical Integrity

A critical implementation detail in the checkout flow is data snapshotting. In a relational database, changing a user’s address usually updates all linked records. However, for an invoice, the address at the time of purchase must be preserved.

The `orders` table implementation includes JSON columns to store a frozen snapshot of the shipping/billing address and product details at the moment of transaction finality. This ensures that future changes to the `user_settings` or `products` tables do not historically alter completed orders, a requirement for auditability.

5.5 The Feedback Loop: Reactive Reconciliation

The final component of the architecture is the feedback loop that reconciles the Optimistic UI with the server’s authoritative state.

5.5.1 The Command-Query Cycle and Write-Path Synchronization

The system implements a cycle of reconciliation to close the loop between the write path and the read path. This cycle relies on a “write-path sync” pattern to safely transition the user interface from a local optimistic state to a globally authoritative state without visual tearing.

This cycle operates through five distinct stages, grounded in the application’s specific tooling:

1. **Command (Optimistic Local Write):** The cycle initiates when the client performs a local mutation via a TanStack DB collection (e.g., `categoriesCollection.insert()`). TanStack DB immediately applies this change to the local in-memory store, triggering a synchronous UI update for zero-latency feedback.
2. **Commit (Authoritative Server Write):** Within the collection's mutation hook (`onInsert` or `onUpdate`), a synchronous `tRPC` mutation is dispatched to the server. The server validates the invariant and executes the operation within a PostgreSQL transaction. Crucially, the server generates a unique transaction ID (`txid`) for this operation and returns it to the client.
3. **Capture:** ElectricSQL's CDC mechanism detects the applied transaction in the PostgreSQL database.
4. **Propagate:** The mutation is pushed down the **HyperText Transfer Protocol** (HTTP) replication stream to all subscribed clients, carrying the original payload and its associated metadata.
5. **Reconcile (State Discard):** This is where the dual-path architecture resolves. The TanStack DB collection hook returns the server-provided `txid` to the local engine. Under the hood, TanStack DB utilizes an `awaitTxId` utility to actively monitor the incoming ElectricSQL replication stream for this specific identifier. Once the authoritative server write (matching the `txid`) arrives via ElectricSQL, TanStack DB safely discards the temporary optimistic local state and seamlessly swaps in the authoritative data.

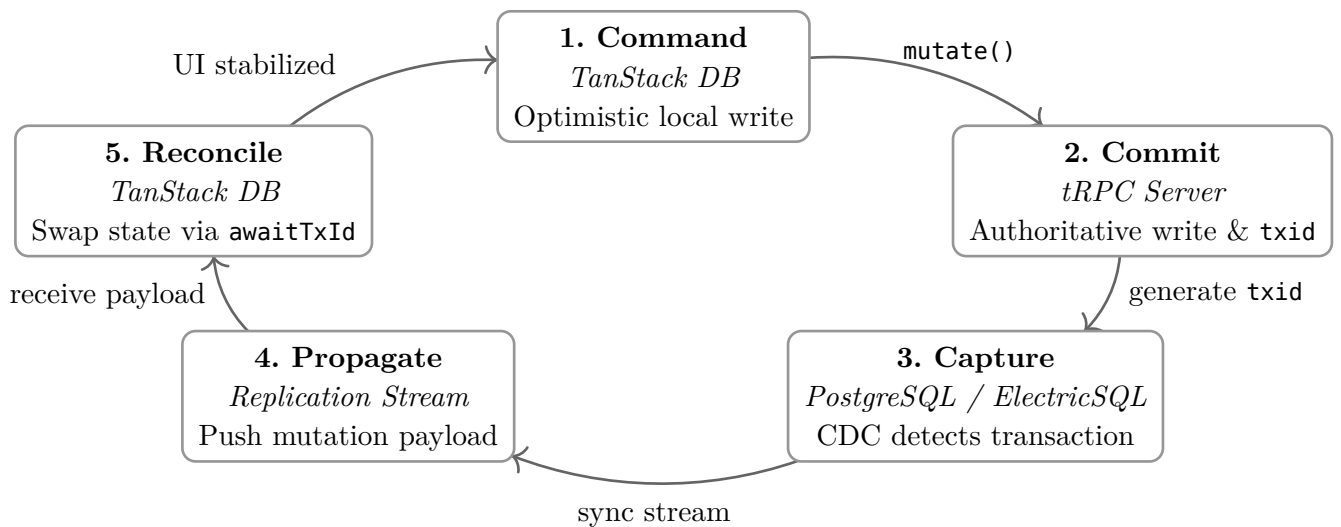


Figure 5.3: The Command-Query Cycle and Write-Path Synchronization

This pattern is explicitly visible in the application's collection definitions, where the bridging of the Optimistic UI and authoritative tRPC mutation occurs:

```
// Source: Bridging optimistic state with authoritative txid
export const categoriesCollection = createCollection(
  electricCollectionOptions({
    // ... shape configuration
    onInsert: async ({ transaction }) => {
      const { modified: newCategory } = transaction.mutations[0];

      // The local optimistic state is already rendered.
      // Now, dispatch the authoritative command via tRPC:
      const result = await trpc.categories.create.mutate({
        name: newCategory.name,
        color: newCategory.color
      });

      // Return the transaction ID so TanStack DB's awaitTxId
      // can monitor the Electric sync stream for reconciliation.
      return { txid: result.txid };
    }
  })
);
```

By returning { txid: result.txid }, the framework guarantees that the UI will not regress or flicker. The local state is maintained exactly until the moment the verified, globally consistent state arrives through the read path, fulfilling the PACELC requirements for a robust local-first application.

Evaluation

Having established the architecture as the primary design result, this chapter evaluates that artifact to answer the second half of the research question: quantifying the specific trade-offs involved in adopting this paradigm.

As established in the Methodology chapter, all performance evaluations were conducted within a controlled simulation environment utilizing Playwright. The software artifact was benchmarked by comparing two distinct architectural paradigms, Local-First and an industry-standard SSR-Hydrated SPA (hereafter referred to as SSR-SPA), across the previously defined device profiles, network constraints, and dataset sizes. To ensure statistical validity, all test scenarios and metric recordings were repeated 30 times for each configuration.

6.1 Performance Benchmarks

This section details the empirical data collected concerning the application’s temporal performance metrics. The following subsections analyze the initial synchronization and load wait times, followed by an evaluation of the latency experienced during localized and remote database interactions.

6.1.1 Initial Synchronization and Load Latency

The initial load latency was quantified by measuring the duration from the “Page Load Start” event to the **L**argest **C**ontentful **P**aint (LCP), defined in this context as the rendering of fully populated, interactive product cards on the UI. For the Local-First architecture, this metric is termed “Initial Sync Time” and encompasses the full background download, parsing, and indexing of the initial database replica. For the SSR-SPA baseline, this represents the standard network fetch, server-side template compilation, and client-side DOM hydration.

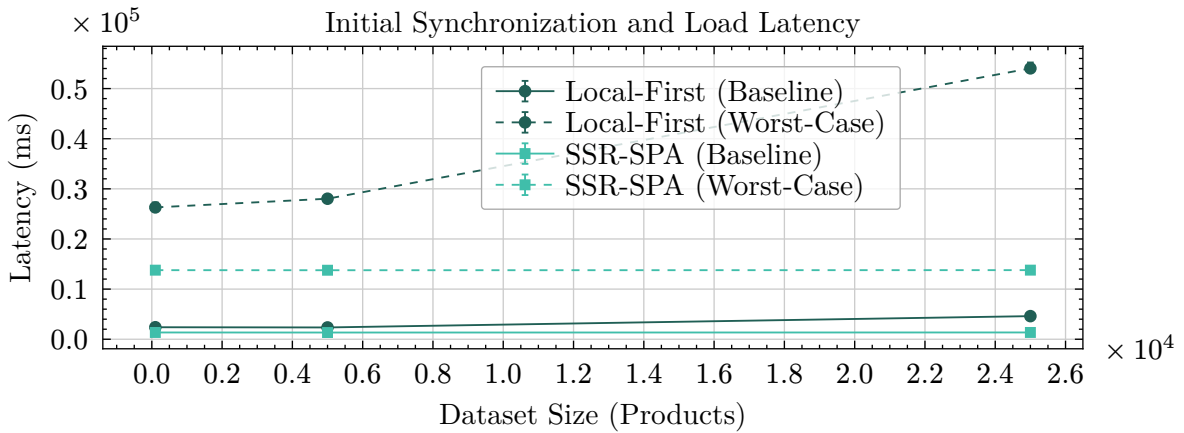


Figure 6.1: Initial Synchronization and Load Latency comparing Local-First vs SSR-SPA across varying dataset sizes. Error bars denote the 5th and 95th Percentile (P95).

As demonstrated in Figure 6.1, across the simulated network conditions, the wait times required to reach an interactive state diverged significantly based on both the architecture and the underlying dataset size.

For the Local-First architecture under Baseline conditions, the wait times scaled proportionately with the catalog size. The Boutique dataset (100 items) required a median sync time of 2394.25 ms, with a P95 latency of 2416.53 ms. Scaling to the Large Retailer dataset (25,000 items) increased this median baseline load time to 4609.42 ms (P95: 5079.24 ms). When subjected to the Worst-Case profile (Slow 3G, Throttled CPU), the Local-First architecture exhibited a severe degradation, requiring a median of 54032.5 ms (P95: 55202.24 ms) to successfully replicate and index the large 25,000-item dataset to the local client before the UI could unblock. Conversely, the SSR-SPA architecture demonstrated load times that were largely invariant to the underlying total dataset size, as the server merely fetches and renders a paginated view. Under Baseline conditions, the median initial load for the Boutique dataset was 1351.23 ms (P95: 1415.87 ms), while the Large Retailer dataset maintained a median of 1354.01 ms (P95: 1385.27 ms); this represents a negligible difference. However, SSR-SPA remained highly susceptible to network degradation. Under the Worst-Case profile, the median initial load time for the Large dataset spiked to 13784.88 ms, with tail latencies stretching to a P95 of 14016.5 ms due to the slow 3G network impeding the delivery of the fully rendered HTML document and subsequent hydration process.

6.1.2 Interaction Latency

Interaction latency was evaluated by measuring the time required to execute complex search query and filter operations directly against the database following the initial application load. For the Local-First artifact, this measured local IndexedDB/CRDT querying. For the SSR-SPA artifact, this measured the round-trip API request and subsequent DOM replacement.

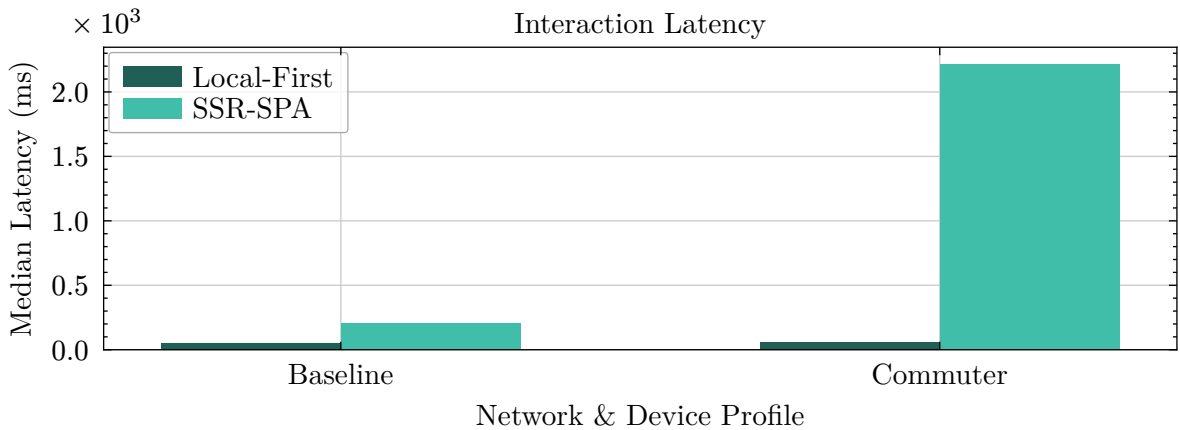


Figure 6.2: Median Interaction Latency comparing Local-First vs SSR-SPA under Baseline and Commuter network profiles for the Mid-Sized Dataset.

As illustrated in Figure 6.2, under the 5,000-item load representing the Mid-Sized Store on the Baseline profile, the median execution time for filter operations was 53.25 ms for the Local-First architecture, keeping tail latencies low with a P95 of 75.96 ms. For the SSR-SPA baseline, the median interaction required 205.85 ms, with a P95 of 213.22 ms.

The architectural divide became most apparent under the Commuter profile, which simulates a high-latency, low-bandwidth Slow 3G network while maintaining standard CPU

performance. Because the Local-First architecture resolves queries against the local device storage, its interaction latency remained inherently unaffected by the network degradation, yielding a median of 61.5 ms and a P95 of 83.92 ms. In strict contrast, the SSR-SPA architecture, despite utilizing TanStack Query to manage state, still mandates a network round-trip for novel state changes and remote filter executions. This resulted in a significantly deteriorated interaction latency under the same Commuter conditions, recording a median of 2213.85 ms and severe tail delays reaching a P95 of 2261.72 ms.

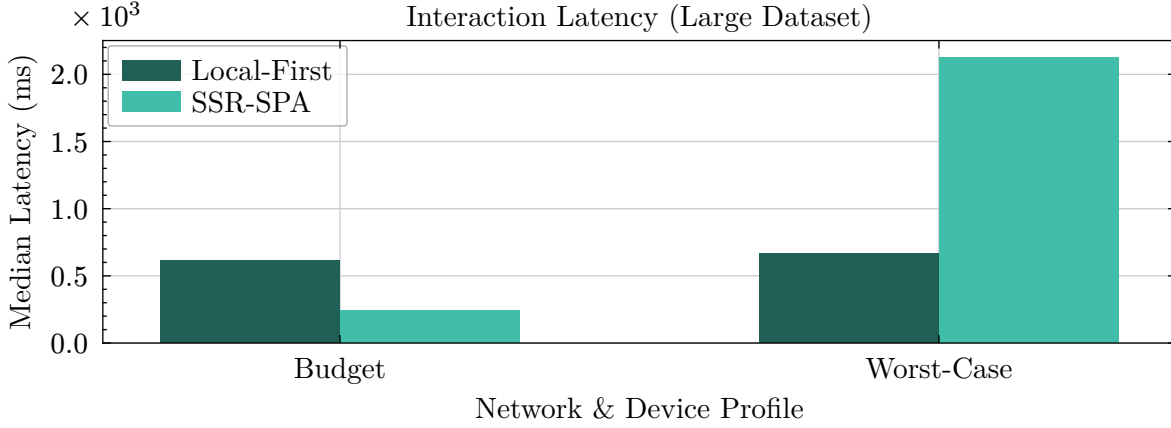


Figure 6.3: Median Interaction Latency comparing Local-First vs SSR-SPA under Budget and Worst-Case profiles for the Large Retailer Dataset.

However, the performance dynamic shifted when evaluating the impact of CPU constraints, particularly when querying the 25,000-item Large Retailer dataset (Figure 6.3). While Local-First won clearly in scenarios where the network is the decisive factor, it relies heavily on client-side processing power. Under the Budget profile, which simulates a throttled CPU but maintains a fast 4G network, the interaction latency on Local-First was noticeably worse than on SSR-SPA. Local-First operations deteriorated to a median latency of 613.4 ms. In contrast, SSR-SPA offloaded the heavy computational querying to the server, returning the response over the fast network with a much faster median latency of 244.85 ms.

When testing the Worst-Case profile, which combines both a throttled CPU and a slow 3G network, the compounding bottlenecks became evident. As shown in Figure 6.3, the SSR-SPA architecture suffered tremendously from the network penalty again, skyrocketing to a median latency of 2124.15 ms. Meanwhile, Local-First remained primarily bottlenecked by the device’s CPU rather than the network, yielding a median latency of 665.85 ms, ultimately outperforming SSR-SPA when both constraints are present.

6.2 Client Resource Utilization

To assess the computational and storage overhead imposed on the local device during the primary data replication and rendering phases, client resource utilization was monitored across both architectural strategies.

6.2.1 Browser Storage Footprint

The browser storage footprint was analyzed by measuring the total client-side storage consumption accumulated within IndexedDB and persistent browser caches during the session. These storage metrics were captured using CDP `Storage.getUsageAndQuota`.

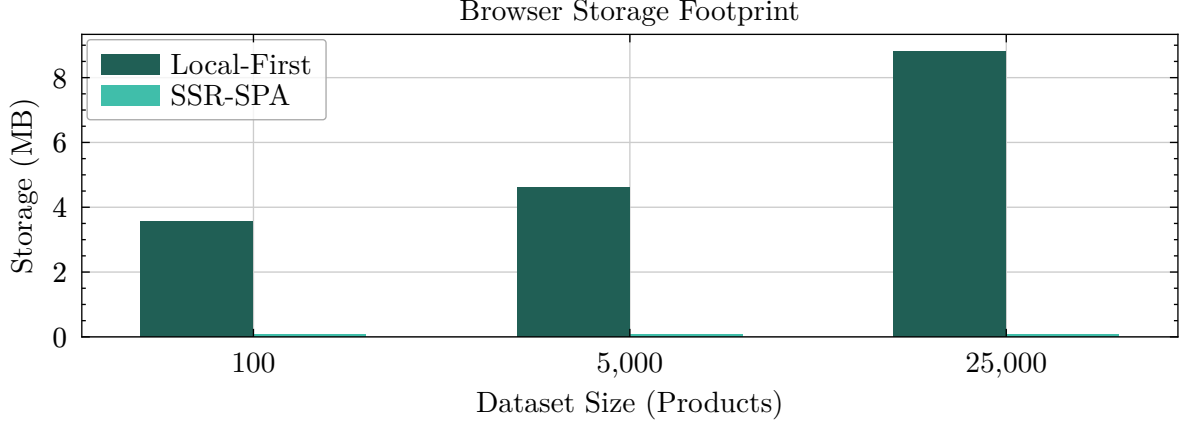


Figure 6.4: Browser Storage Footprint comparing Local-First vs SSR-SPA across varying dataset sizes.

By design, the SSR-SPA baseline offloads persistent data to the remote PostgreSQL infrastructure and restricts query caching entirely to memory via TanStack Query. Consequently, its persistent local storage footprint remained strictly nominal at 0 MB across all dataset sizes and testing profiles, as shown in Figure 6.4.

Conversely, measurements of the Local-First architecture revealed a linear correlation between the catalog size and local storage requirements. Excluding binary assets such as images, the storage footprint for the Local-First artifact reached a total of 8.81 MB for the 25,000-item Large Retailer dataset.

6.2.2 Client Memory Tax

Client resource utilization was monitored by recording JavaScript (JS) Heap Memory Snapshots (MB) during peak execution phases.

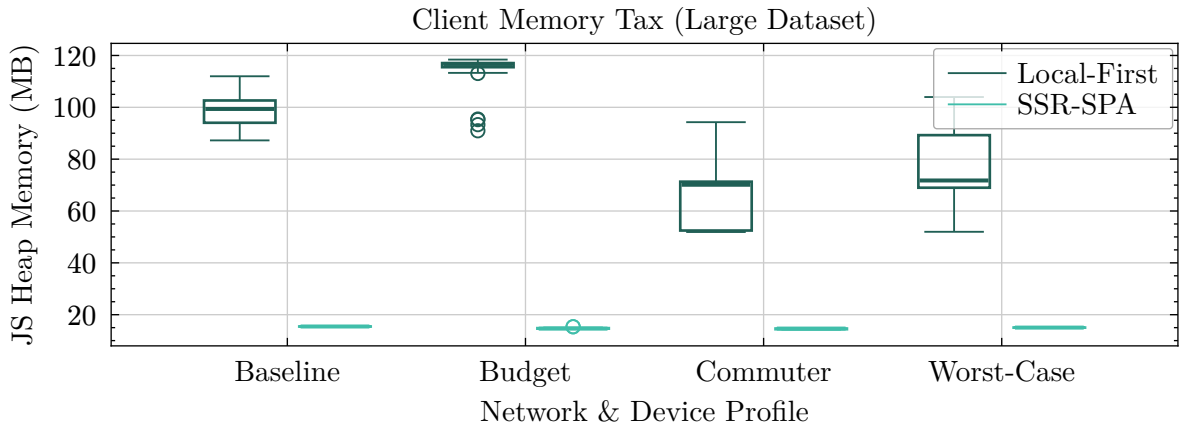


Figure 6.5: Client Memory Tax (JS Heap) during the intensive initial synchronization and rendering of the Large Retailer dataset.

During the intensive initial synchronization and indexing phase of the Large Retailer dataset (visualized in Figure 6.5), the Local-First architecture required substantially higher memory allocations to construct the local CRDT structures and process the incoming data stream. The recorded Heap Memory consumption for Local-First ranged from an average of 75.32 MB (P95: 92.95 MB) under the Worst-Case profile to an average of 113.46 MB (P95: 117.47 MB) under the Budget profile.

In contrast, while the SSR-SPA architecture must allocate heap memory to support hydration JS and maintain the internal TanStack Query cache layer, it still preserved a leaner and more predictable footprint overall. The SSR-SPA artifact’s heap utilization remained steady, averaging between 14.74 MB (P95: 15.22 MB) and 15.44 MB (P95: 15.51 MB) across all profiles.

6.3 Network Utilization and Efficiency

This section analyzes the system’s data transfer characteristics to determine overall network efficiency. It evaluates the bandwidth consumed during typical multi-page navigation sequences and measures the size of the background state reconciliation payloads.

6.3.1 Bandwidth Consumption During Navigation

Bandwidth consumption was evaluated by tracking the total kilobytes (KB) of data transferred during a standardized simulated user journey, which progressed sequentially from the Homepage to a Search Page and finally to an individual Product Page.

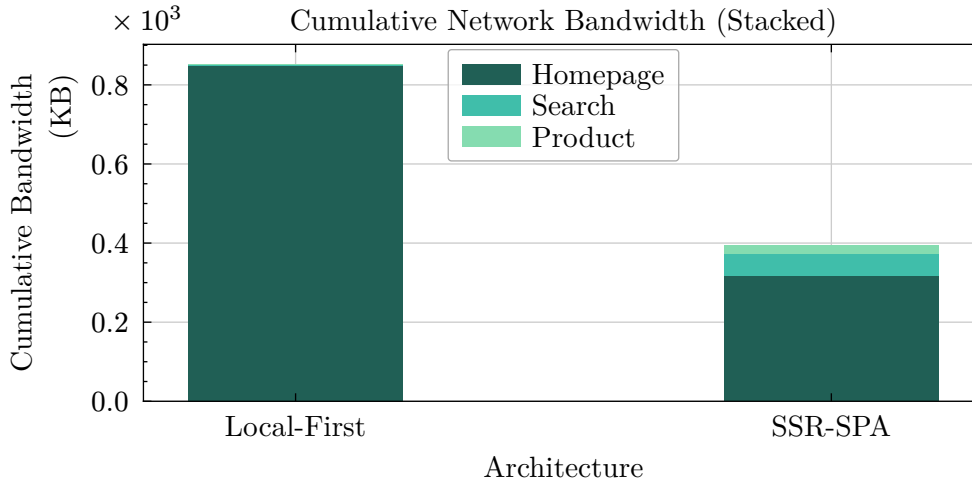


Figure 6.6: Cumulative Bandwidth Consumption during a typical user journey (Homepage -> Search -> Product) with the Mid-Sized Store.

The data transmission profiles of the two architectures illustrate a stark inversion of upfront versus incremental costs, clearly depicted in Figure 6.6. To ensure full offline capability, the Local-First architecture pre-loads the entire data graph as soon as the first route loads. Consequently, the true upfront cost was incurred immediately upon the initial cold load of the Homepage. Bootstrapping the local replica with the complete dataset (5,000 products) resulted in an initial transfer amounting to 847.99 KB. However, once this local database was populated, a drastic drop-off in network reliance was observed. Navigating to the Search page triggered a negligible supplementary transfer of only 3.38 KB. The

subsequent transition to the individual Product Page required an average of just 0.02 KB, making further navigations virtually cost-free over the network.

In contrast, the SSR-SPA baseline exhibited a lower initial entry cost, consuming 318.19 KB upon loading the Homepage. Because the SSR-SPA baseline operates without a fully localized data graph, subsequent route changes necessitated component-level data fetching (e.g. Product details). Navigation to the Search page consumed 53.25 KB, while navigating to the individual Product page necessitated an additional 22.68 KB transfer.

6.3.2 Delta Update Payload Efficiency

To measure the efficiency of the background replication stream within the Local-First context, a test was conducted in which a single item’s price was modified directly on the server-side database while the client maintained an active session.

The resulting granular delta payload received by the Local-First client to reconcile this localized state modification amounted to a raw byte size of 326 bytes. No directly comparable metric was isolated for SSR-SPA, as state updates in the SSR-Hydrated SPA architecture are intrinsically tied to either full document replacements or larger `json` API refetches triggered by TanStack Query’s invalidation mechanics, which inherently scale with the query configuration rather than strictly the raw delta mutation.

6.4 Distributed State Consistency and Conflict Resolution

The following subsections evaluate the architectures’ abilities to maintain data integrity during concurrent, partitioned updates. This includes an analysis of conflict resolution strategies during simulated split-brain scenarios and the time required for clients to achieve eventual consistency upon reconnection.

6.4.1 Conflict Scenario A: Execution and Gatekeeping

To evaluate the handling of semantic conflicts under partitioned network conditions, a standardized “Split-Brain” scenario was executed. The central database recorded an inventory of exactly one unit for a specific item. Client A transitioned to an offline state and added the item to their local cart. Concurrently, Client B remained online and successfully purchased the same final item, reducing the authoritative server inventory to zero.

When Client A reconnected to the network and attempted to execute a checkout operation, the systems diverged by design. The Local-First CRDT initially allowed the cart addition during the offline phase, accepting the optimistic write. However, upon executing the checkout mutation via the synchronous RPC layer, the server evaluated the `stock >= 0` invariant. The server strictly rejected the transaction, returning an `HTTP 409 Conflict`.

For the SSR-SPA baseline, this split-brain scenario could not materialize in the same manner. Because the SPA architecture strictly couples authoritative state writes and validations to an active network connection, Client A was immediately blocked from adding the item to the cart the moment the network dropped, preventing the semantic conflict at the point of origin but entirely disabling offline interactivity.

6.4.2 Conflict Scenario B: Structural Concurrency

The test utilized Playwright’s isolated Browser Contexts to simulate two independent, concurrent sessions (“User A” and “User B”). The scenario began with a collaborative cart containing a single folder that held one item. Both contexts were deliberately taken offline via Playwright’s network routing features before they independently modified the nested structural tree. Specifically, User A added a new item to the folder, while User B concurrently deleted that folder. The network connection for both contexts was then restored to observe the reconciliation process.

This scenario explicitly evaluated Requirement 2 (Mathematical Convergence). While flat lists can rely on simple “Last-Writer-Wins” logic, nested cart structures risk generating cycles or orphaned data during concurrent structural mutations. To validate architectural claims regarding structural complexity and robust offline collaboration, the test evaluated three deterministic assertions upon network restoration.

The empirical results aligned with the expected mathematical properties of the underlying Yjs data structures, augmented by application-level conflict resolution:

1. **Stability:** Passed. Neither browser context threw a JS runtime exception during the merge. The local CRDTs replicas processed the remote byte arrays without causing infinite loops or memory faults.
2. **State Parity:** Passed. Both Context A and Context B successfully converged to an identical, deterministic structural state immediately following the peer-to-peer synchronization exchange.
3. **Resolution Logic:** Verifiable. The test confirmed that the newly added item was orphaned rather than deleted. While native CRDT behavior often cascades deletions to child elements, the application employs deliberate self-healing logic that catches orphaned items during reconciliation (detailed in Appendix B-F). Consequently, the final reconciled state resulted in the target folder (and its original item) being removed, while the cart exclusively contained the new item added offline by User A.

6.4.3 Time to Consistency (TTC)

For the Local-First architecture, the TTC metric tracked the duration elapsed from the exact moment of client network reconnection to the successful background reconciliation of the remote state. Across the iterated test runs, the time required for the background sync engine to fetch the updated delta state and for the local CRDT to apply the remote operations averaged 45.47 ms (P95: 118.07 ms) under Baseline conditions, and 96.52 ms (P95: 141.65 ms) on Budget hardware. The SSR-SPA baseline does not generate an equivalent TTC metric, as it lacks a localized offline operational state to reconcile upon reconnection.

6.4.4 Server-Authoritative Gatekeeping (Malicious Payload Interception)

To verify the system’s resilience against client-side data manipulation, a security-focused test was conducted by simulating a malicious payload interception during the checkout sequence. In this scenario, the client’s network request to the tRPC endpoint was intercepted, and the payload was artificially modified to force the item’s price to 0, simulating a bad actor attempting to bypass standard pricing logic.

The empirical result of this test demonstrated that the manufactured price data submitted by the simulated malicious client was completely ignored by the backend. The server successfully maintained transactional integrity by calculating the final checkout price server-side, independently referencing the authoritative product database rather than trusting the client-provided input. Consequently, the modified payload failed to alter the final order cost, validating the system’s strict enforcement of server-authoritative gatekeeping and its immunity to client-side price injection attacks.

6.5 Offline Feature Availability

To systematically verify the artifacts’ functional resilience during network isolation, a binary audit of core application features was conducted with the simulated network connection fully disabled.

The audit confirmed the isolation of the read-path and the write-path functionality within the Local-First paradigm. Under complete offline conditions, catalog browsing, product filtering, and complex search queries retained `Full` functionality, serving data directly from the local IndexedDB replica. Modifying the shopping cart operated with `Local` functionality, successfully committing operations to the local Yjs document. In contrast, the SSR-SPA artifact recorded a `Blocked` status across all read and write features during network isolation, failing immediately to a static offline boundary.

Discussion

The primary objective of this research was to investigate the architectural viability and performance implications of applying a “hybrid local-first” paradigm to e-commerce. Specifically, this study evaluates how the constructed “Three-Pillar” architecture addresses the tension between offline capabilities and strict inventory consistency. Building upon existing local-first principles, traditionally rooted in collaborative software, this research assesses their practical feasibility within the constraint-heavy domain of relational retail data. The empirical findings provide a nuanced perspective on the trade-offs inherent in distributed systems design, effectively demonstrating how the PACELC framework applies to real-world e-commerce scenarios.

7.1 Navigating the PACELC Trade-offs: Latency vs. Client Resources

The performance benchmarks clearly illustrate that the local-first architecture effectively eliminates network latency from the user’s critical path during navigation, fulfilling the “No Spinners” ideal [1]. However, this achievement reflects a fundamental relocation in systemic bottlenecks rather than an outright elimination of computational cost.

The Paradigm of Shifting Bottlenecks

The interaction latency results demonstrate a stark contrast in hardware dependency. While the SSR-Hydrated SPA architecture is inherently network-bound, the Local-First architecture is fundamentally CPU-bound. By resolving queries against local storage, Local-First effectively neutralizes network degradation, vastly outperforming the SSR-SPA baseline in query execution under poor network conditions (61.5 ms vs. 2213.85 ms).

Hardware Constraints

Conversely, the reliance on client-side processing introduces new performance liabilities. When simulated on throttled CPUs, the SSR-SPA baseline outperformed Local-First in interaction latency. This confirms that local-first systems offload the heavy computational burden of indexing and querying the database directly onto the client device, penalizing users on lower-tier hardware (613.4 ms vs. 244.85 ms). This empirical finding aligns closely with recent local-first studies. For instance, Zavalysyn et al. observed a similar hardware dependency when comparing local-first smart home hubs to cloud-only alternatives, noting that cloud computation was 33% to 55% faster than local processing on constrained devices [61]. Kleppmann et al. also documented performance and memory bottlenecks when operating CRDTs on client devices, as the structures accumulate large change histories that heavily tax local resources [1]. Furthermore, Siffre et al. highlighted that while local-first architectures drastically reduce network transmission costs, they inherently shift the energy and computational burden back to the client’s CPU [62]. Therefore, the performance penalty on throttled hardware reflects a fundamental paradigm shift: trading network-bound bottlenecks for CPU-bound constraints.

The Cost of Initial Synchronization

The most significant barrier to adoption identified in the results is the initial load latency. Replicating the large retailer dataset imposed a severe upfront data transfer and processing cost. This substantial initial load time (up to 55202.24 ms), combined with a nearly seven-fold increase in memory residency compared to the SSR-SPA baseline (113.46 MB vs. 15.44 MB), suggests that scaling local-first architectures necessitates aggressive, query-based partial replication (such as server-enforced shape syncing) to remain viable on mobile devices.

7.2 Resolving the Invariant Problem and Authoritative Gatekeeping

A central theoretical hurdle identified in the literature was the “Invariant Problem”, the inability of standard CRDTs to enforce semantic correctness, such as preventing negative inventory balances during offline purchasing [9], [12].

Other researchers have proposed various workarounds to this limitation. Schiefer et al. proposed a “Forking Histories” model, which avoids rejecting writes by allowing the application state to branch into multiple co-existing histories during a conflict, leaving the logical reconciliation to the user at a later time [17]. Alternatively, automated verification systems like ConLoc (Köhler et al.) and LoRe (Haas et al.) attempt to solve this by statically analyzing the code and automatically injecting strong consistency protocols, such as distributed token-based locks, only for the operations that risk violating invariants [9], [12].

While these approaches maintain data integrity, they introduce significant UX and architectural complexity. Forking histories force end-users to manually resolve merge conflicts, a paradigm familiar to programmers but highly detrimental to a frictionless e-commerce checkout experience [1]. Meanwhile, automated locking mechanisms reintroduce network blocking into the local-first flow, potentially stalling the UI while waiting to acquire distributed consensus [1].

The “Three-Pillar” architecture effectively bypassed this limitation not by attempting to achieve formal verification of business logic on the client, nor by exposing complex version histories to the buyer, but by adopting a pragmatic CQRS approach.

Contrasting with the Escrow Pattern

As outlined in the State of Research, implementing an Escrow (or Reservation) pattern would theoretically enable true offline checkout capabilities by pre-allocating a pool of inventory rights directly to client replicas. However, critically assessing this approach reveals that it shifts the consistency bottleneck rather than eliminating it. The implemented architecture intentionally avoids the Escrow pattern due to its heavy “rights management” overhead and the severe economic risk of “inventory fragmentation”, a scenario where vital stock becomes leased and temporarily locked on disconnected, unreachable devices. However, while the Escrow pattern was deemed too risky for this baseline prototype, it remains the most viable pathway for future extensions aiming to enable fully offline checkouts, a concept explored further in Chapter 8.

The Practical Compromise

To avoid these complexities, making the checkout an online-only process serves as the practical solution. By explicitly routing checkout mutations through a synchronous, server-authoritative RPC (tRPC) channel, the system establishes a strict Trust Boundary.

- While this approach fundamentally sacrifices the local-first ideal of complete offline availability specifically for the final checkout action, it proved advantageous for the reality of e-commerce. It preserves the “No Spinners” ideal for catalog browsing and cart management, while guaranteeing strict serializability for financial operations without requiring novel CRDT verification algorithms.
- Furthermore, the security evaluation demonstrated that the architecture is resistant to client-side payload manipulation (e.g., price spoofing). Because the backend calculates final transaction costs independently of the client’s local state, the system successfully enforces server-authoritative gatekeeping.

This validates that Strict Serializability (PC/EC) can be seamlessly integrated into a predominantly Highly Available (PA/EL) application without violating the integrity of the broader data model.

7.3 The Local-First Spectrum: Adapting Ideals to E-Commerce

The foundational literature introduces local-first software through seven strict ideals [1]. However, evaluating this prototype reveals that the local-first paradigm is not a rigid binary classification but rather a flexible spectrum. Many contemporary tools label themselves “local-first” while selectively implementing only the principles that align with their specific domain requirements. For instance, commercial synchronization frameworks like RxDB [40] provide robust offline availability, yet lock essential persistence modules behind proprietary paywalls, effectively compromising the ideals of “The Long Now” and ultimate user data ownership. Similarly, infrastructure like ElectricSQL [41] prioritizes zero-latency interactions but fundamentally tethers the local state to a central, authoritative PostgreSQL server. This effectively bypasses pure peer-to-peer collaboration and end-to-end encryption in favor of maintaining relational data integrity.

This prototype intentionally diverges from the final three local-first ideals, as strict adherence proved incompatible with or irrelevant to the constraints of digital retail.

First, the ideals of “The Long Now” and “You Retain Ultimate Ownership and Control” are fundamentally rooted in collaborative creation software where users generate valuable, long-term artifacts [1]. In contrast, the data within an e-commerce context largely consists of transient shopping carts. While users could theoretically retain ownership by dumping their persistent client-side storage (e.g., IndexedDB), this process is highly cumbersome and fails to capture the intended spirit of the ideal. More importantly, users do not associate a high degree of personal ownership with an ephemeral list of retail items, making these specific ideals minimally beneficial to the core user experience.

Second, the prototype did not implement the ideal of “Security and Privacy by Default” through end-to-end encryption. The absolute necessity for transactional integrity dictates that the central server must act as an authoritative gatekeeper. The backend requires a

clear, unencrypted view of the cart contents to validate inventory levels, calculate taxes, and execute the final financial transaction. Consequently, an unencrypted, server-authoritative trust boundary is a necessary compromise that supersedes strict privacy ideals in this domain.

7.4 Structural Concurrency and Collaborative Resilience

The elevation of the shopping cart to a multi-user collaborative workspace served as a rigorous stress test for SEC. Moving beyond simple flat lists, the integration of Yjs utilizing fractional indexing facilitated complex structural mutations without index collisions.

During the simulated structural concurrency conflict (simultaneous folder deletion and item movement by disconnected clients), the system successfully achieved state parity without generating orphaned data nodes or infinite loops. This deterministic reconciliation proves that modern CRDTs, when coupled with a reliable CDC replication stream like ElectricSQL, provide a robust mathematical foundation for complex, offline-first collaboration.

7.5 Network Utilization and Offline Functionality

Finally, the results highlight a paradigm shift in network economics. Traditional web architectures require continuous, incremental data transfers that scale linearly with UI complexity. The Local-First model demonstrated an inverse data-consumption profile: after an initially heavy bootstrap phase, subsequent route navigations consumed a negligible amount of supplementary data (326 bytes). This extreme bandwidth reduction facilitates true network optionality. The feature availability audit confirmed that core catalog browsing and cart modification remained fully functional during complete network isolation.

7.6 Limitations

While the “Three-Pillar” architecture successfully demonstrates the viability of hybrid local-first e-commerce, several limitations within the study’s scope must be acknowledged:

Simulation vs. Reality

The empirical evaluations were conducted using Playwright to simulate CPU throttling and degraded network profiles. While standard for automated benchmarking, these synthetic constraints cannot perfectly emulate the volatility of real mobile environments, including thermal throttling, browser garbage collection pauses, and intermittent packet loss.

Client Resource Drain

The local-first paradigm inherently demands higher CPU and memory allocation to parse CRDTs and maintain IndexedDB replicas. Dealing with large local datasets in IndexedDB requires heavy transaction management that can freeze the user interface and cause significant application delays. On real-world mobile devices, this sustained computational overhead directly translates to increased battery consumption, a critical UX metric not

quantified in this study, but empirically covered in recent research by Siffre et al [62]. Their measurements confirm that shifting computation to the client device to process tasks locally consumes significantly more battery energy than traditional SaaS offloading. However, it is worth noting that while the individual device incurs a battery penalty, modern smartphone processors are increasingly mitigating this local drain, ultimately making the total system-wide energy consumption (when factoring in saved network and server costs) highly competitive.

Upper Bounds of Scalability

This study stress-tested the system with a dataset of 25,000 items. However, browser storage quotas and the JavaScript heap's **Out of Memory** (OOM) limits present a definitive ceiling. The performance impact of approaching these limits was not explored at an enterprise scale (e.g., 100,000+ items).

While these constraints define the boundaries of the current empirical data, they also illuminate necessary pathways for the continued evolution of distributed retail systems. A detailed exploration of these future research vectors and the long-term outlook for the local-first paradigm is provided in the subsequent chapter.

7.7 Conclusion

The evaluation indicates that a hybrid consistency model presents a viable approach for applying local-first principles to e-commerce. While it demands increased client-side memory and compute resources, and trades initial load time for subsequent speed, it demonstrates a practical compromise between instantaneous offline availability and strict transactional integrity.

Outlook

The implementation and evaluation of the “Three-Pillar Architecture” in this thesis demonstrate that a hybrid local-first approach is not only technically feasible but offers some advantages for e-commerce performance and UX. However, as with any nascent architectural paradigm, the transition from a research prototype to a production-grade industrial standard requires addressing remaining hurdles in scalability, security, and developer tooling. This chapter outlines the potential adoption landscape, proposes technical extensions to the current artifact, and identifies critical avenues for future research.

8.1 Industry Adoption and Market Viability

While this project initially focused on a general-purpose **Business-to-Consumer** (B2C) webshop, the empirical results indicate that asserting its viability for general B2C commerce without solving the initial bootstrap cost is highly problematic. Consequently, the true economic implications of the local-first paradigm currently extend much more naturally into B2B and industrial contexts.

High-Latency Markets and the “Next Billion Users”

The most immediate beneficiaries of this architecture are markets with developing digital infrastructure. As of 2024, a significant percentage of the global population in regions such as Southeast Asia and Sub-Saharan Africa accesses the web primarily via unstable mobile networks [63]. In these contexts, latency is not merely a nuisance but a barrier to entry [64]. The local-first model, which decouples interaction speed from network quality, addresses this “economic cost of latency,” theoretically allowing global retailers to penetrate markets where traditional SSR or industry standard SSR-Hydrated SPA fail to convert due to connectivity friction.

However, this potential is currently obstructed by a severe B2C ‘first-load’ barrier. While the local-first model successfully eliminates latency during active use, it does so at the expense of a massive upfront synchronization cost. Until this initial hurdle, such as the nearly 60-second load time observed on 3G networks, is mitigated, adopting this architecture for general B2C retail in high-latency regions is unlikely to improve initial conversion rates over these conventional architectures.

B2B and Field Operations

Perhaps the strongest use case lies in B2B commerce and logistics. Field sales representatives often operate in environments with poor connectivity (e.g., warehouses, hospital basements, remote industrial sites). A local-first application allows a sales representative to browse a catalog of 25,000+ SKUs, build complex orders, and manage client profiles entirely offline, syncing only when connectivity is restored.

- **Target Demographic:** Wholesalers, medical device suppliers, and automotive parts distributors.
- **Operational Efficiency:** Eliminating the “loading spinner” for internal tools can recover significant man-hours. For a logistics company with 1,000 employees performing 50 interactions a day, saving 2 seconds per interaction translates to roughly 28 hours of recovered productivity daily.

8.2 Functional Extensions and Conflict Resolution

The current prototype relies on a strict separation between the Collaborative Local State (Shopping Cart) and the Authoritative Write (Checkout). Future iterations could blur these lines to enhance utility.

Offline-Capable Authoritative Writes

Currently, checkouts require an active connection. Future iterations could enable asynchronous, offline writes using optimistic rights management. Instead of global server-side stock checks, the system could utilize an Escrow Pattern, pushing bounded inventory “allowances” to the local TanStack DB. Offline transactions would embed lease identifiers and timestamps, which the server validates upon reconnection, seamlessly triggering rollbacks for stale leases. To securely manage these complex local revocations, **WebAssembly** (Wasm) can run backend verification logic directly in the browser, serving as an accurate optimistic filter before server reconciliation.

Deferred Payment Processing

Shifting payments offline introduces severe regulatory friction. For new customers, synchronous SCA challenges are impossible offline [32], [33], [34], leaving only high-risk alternatives like invoice or unverified SEPA mandates [65], [66, Article 5c]. For existing customers, offline checkouts can leverage vaulted tokens via **Merchant-Initiated Transactions** (MITs) [67], [68]. However, delayed MITs risk card expiration or “soft declines.” Mitigating this friction requires **3D Secure 2.0** (3DS2) Decoupled Authentication, allowing out-of-band mobile banking approvals upon reconnection [34]. Pan-European strategies must also account for regional **Alternative Payment Methods** (APMs) (e.g., iDEAL, Swish) [31, (4)], [69], which operate under distinct legal frameworks and may not universally support delayed offline capture or MITs in the same manner as card networks. Consequently, the most viable offline checkout environment remains B2B, where pre-approved credit lines (Net-30/60) [70] inherently bypass real-time authorization and SCA requirements [31, (73)], [34], [71, Article 17].

Semantic Conflict Resolution

Current synchronization handles structural conflicts, but arbitrary resolution (e.g., **Last-Write-Wins** (LWW) overwriting concurrent quantity edits) fails with semantic conflicts. Future systems should implement a “Conflict Inbox” for manual resolution, shifting from eventual consistency to eventual correctness, similar to Git [17].

8.3 Avenues for Future Research

Predictive Partial Replication

A core challenge in local-first engineering is deciding what data to sync: syncing everything is infeasible, but syncing too little breaks offline functionality. Future research should investigate Predictive Partial Replication using machine learning. By analyzing user behavior patterns (e.g., “Users who view Category A often view Category B”), a system could probabilistically pre-fetch data “Shapes” before the user explicitly requests them. This would minimize the initial download footprint while maximizing the cache hit rate of the local database.

Privacy-Preserving Edge Intelligence

Traditional e-commerce recommendation engines require sending user activity logs to a central server, raising privacy concerns (**General Data Protection Regulation (GDPR)**). In a local-first architecture, the entire product catalog and the user’s interaction history reside on the device. Research could focus on implementing lightweight recommendation algorithms (**Edge Artificial Intelligence (AI)**) that run entirely in the browser. This would allow for highly personalized shopping experiences without user behavioral data ever leaving the client device, offering a competitive advantage in a privacy-conscious market.

“Local-First Suitability Framework”

Finally, there is a lack of formalized decision frameworks for software architects. Not every application benefits from local-first; for some, the complexity overhead outweighs the UX gains. A proposed research contribution would be the creation of a Local-First Suitability Framework, a scored heuristic model evaluating factors such as “Write Contention Rate,” “Offline Criticality,” and “Data Mutability” to scientifically determine when and for what features a local-first architecture is the optimal design choice.

Conclusion

This thesis investigated the architectural viability and performance implications of a “hybrid local-first” paradigm in e-commerce, specifically addressing the trade-off between offline system capabilities and the strict data consistency required for transactional integrity. Motivated by the limitations of traditional, server-authoritative cloud applications in unstable network environments, the research aimed to bridge the gap between the low-latency characteristics of local-first software and the strict invariant constraints of digital retail.

Using a Design Science Research (DSR) methodology, this study guided the synthesis and evaluation of a “Three-Pillar” hybrid architecture. Grounded in the theoretical boundaries of the PACELC framework, the resulting software artifact decomposed consistency requirements. It applied an Available/Low-Latency (PA/EL) model for catalog browsing and collaborative cart management via partial replication and CRDTs, while enforcing a Partition-tolerant/Consistent (PC/EC) model for the checkout process via a synchronous, server-authoritative RPC layer.

9.1 Key Findings and Contributions

The empirical evaluation of the software artifact, benchmarked against a functionally equivalent SSR-Hydrated SPA baseline, provided several insights into the practical trade-offs of the local-first paradigm.

9.1.1 The Paradigm of Shifting Bottlenecks: Network vs. Client Resources

The results indicated that the local-first architecture reduces network latency on the user’s critical path during standard navigation, minimizing visible loading states. By querying data from an embedded local replica, the system maintained low interaction latency even under network degradation, showing lower latency than the SSR-SPA baseline.

However, this architectural choice relocates the computational load from the network to the client device’s CPU and memory. The study quantified a performance penalty on throttled hardware, confirming that local-first systems offload data indexing and CRDT management onto the user’s machine. As noted in the discussion (Chapter 7), this sustained computational overhead directly translates to increased battery consumption for the mobile device. Furthermore, the architecture successfully mitigates network latency only if the user possesses a sufficiently powerful CPU; on constrained hardware typical of budget devices, interaction latency degraded beyond that of the server-centric baseline.

Most notably, a critical trade-off of this approach is the initial synchronization phase for large datasets: upfront load latencies increased significantly, reaching nearly 60 seconds on slow 3G networks (see Section 6.1.1). This presents a severe “first-load barrier” that makes the architecture’s immediate viability for general B2C commerce highly problematic in its current state. This initial performance penalty is also accompanied by a larger browser storage footprint compared to the traditional SSR-SPA application.

9.1.2 Resolving Invariant and Regulatory Constraints through CQRS

A theoretical challenge identified in the literature is the “Invariant Problem”: the inability of standard CRDTs to enforce application-specific semantic correctness, such as preventing negative inventory balances during concurrent offline purchases. [9]

Beyond the mathematical necessity of preventing negative inventory, this research highlighted that strict financial regulations equally force the adoption of a hybrid architecture. The legal requirements of European PSD2 regulations, Strong Customer Authentication (SCA), and PCI-DSS compliance structurally prevent fully offline B2C electronic payments [32], [33], [34].

This research demonstrated that formal client-side verification or user-facing “Forking Histories” can be avoided in this context. Instead, the “Three-Pillar” architecture adopted a CQRS approach to satisfy both mathematical and regulatory boundaries. By routing critical checkout mutations through a synchronous tRPC channel, the system established a server-authoritative trust boundary. This validation maintained strict serializability for financial operations, prevented semantic conflicts, and showed resilience against client-side payload manipulation.

9.1.3 The Local-First Spectrum in Practice

This research established that the local-first paradigm operates as a spectrum rather than a rigid set of binary rules. While the architecture successfully adopted core principles for offline availability, it intentionally compromised on ideals such as long-term data ownership, ultimate user control, and default end-to-end encryption. These deviations were necessary to accommodate the transient nature of shopping carts and to allow the unencrypted, server-authoritative validation required for secure e-commerce transactions.

9.1.4 Network Efficiency and Offline Autonomy

The data transmission profiles revealed a shift in network utilization. While the initial bootstrap phase of the local-first system required substantial data transfer, subsequent route navigations consumed minimal supplementary bandwidth. This reduced bandwidth reliance facilitated offline capabilities, allowing core features like catalog browsing, search queries, and cart modifications to remain functional during network isolation. Additionally, the integration of Yjs utilizing fractional indexing demonstrated that CRDTs provide a reliable mechanism for offline-first structural collaboration, reconciling concurrent nested mutations.

9.2 Implications for Software Engineering and E-Commerce

The findings of this thesis present considerations for the design of distributed retail platforms. The trade-off between applications that prioritize offline speed and those that prioritize strict consistency can be mitigated. The proposed hybrid architecture demonstrates that e-commerce platforms can partially decouple the user experience from underlying network instability.

While theoretically relevant for high-latency environments with developing digital infrastructure, this study reveals a critical caveat: the reliance on client-side processing means it only benefits users with devices capable of handling the heavy local database indexing. Due to the severe first-load barrier and the hardware constraints of budget devices, this architecture is currently most viable for operational workflows in B2B commerce and field logistics. In these contexts, representatives can execute procurement tasks in environments with limited or no internet connectivity, where the initial sync cost can be amortized over a long-lived professional session.

9.3 Final Remarks

Transitioning from a centralized, server-authoritative model to a hybrid framework introduces architectural complexity and places increased demands on edge-device hardware. The necessity for partial replication, client-side memory management, and server-side validation requires careful engineering.

However, the barriers to implementation are diminishing as the tooling ecosystem for local-first development matures through projects like ElectricSQL and Yjs. By utilizing hybrid consistency models, software architects can combine the responsiveness of local-first architectures with the transactional integrity required by e-commerce. This approach ultimately offers a method to improve user experience across varying network conditions.

Bibliography

- [1] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, “Local-First Software: You Own Your Data, in Spite of the Cloud,” in *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, in Onward! 2019. New York, NY, USA: Association for Computing Machinery, Oct. 2019, pp. 154–178. doi: [10.1145/3359591.3359737](https://doi.org/10.1145/3359591.3359737).
- [2] A. Rotem-Gal-Oz, “Fallacies of Distributed Computing Explained,” *Doctor Dobbs Journal*, Jan. 2008, Accessed: Feb. 01, 2026. [Online]. Available: https://www.researchgate.net/publication/322500050_Fallacies_of_Distributed_Computing_Explained
- [3] P. Bailis, A. Davidson, A. Fekete, A. Ghodsi, J. M. Hellerstein, and I. Stoica, “Highly Available Transactions: Virtues and Limitations (Extended Version).” Accessed: Feb. 16, 2026. [Online]. Available: <http://arxiv.org/abs/1302.0309>
- [4] G. Caso *et al.*, “The Chronicles of 5G Non-Standalone: An Empirical Analysis of Performance and Service Evolution,” *IEEE Open Journal of the Communications Society*, p. 1, Jan. 2024, doi: [10.1109/OJCOMS.2024.3499370](https://doi.org/10.1109/OJCOMS.2024.3499370).
- [5] M. Basalla, J. Schneider, M. Luksik, R. Jaakonmäki, and J. vom Brocke, “On Latency of E-Commerce Platforms,” *Journal of Organizational Computing and Electronic Commerce*, vol. 31, pp. 1–17, Jan. 2021, doi: [10.1080/10919392.2021.1882240](https://doi.org/10.1080/10919392.2021.1882240).
- [6] M. Kleppmann, *Designing Data-Intensive Applications*. Accessed: Feb. 06, 2026. [Online]. Available: <https://www.oreilly.com/library/view/designing-data-intensive-applications/9781491903063/>
- [7] D. Abadi, “Consistency Tradeoffs in Modern Distributed Database System Design: CAP Is Only Part of the Story,” *Computer*, vol. 45, no. 2, pp. 37–42, Feb. 2012, doi: [10.1109/MC.2012.33](https://doi.org/10.1109/MC.2012.33).
- [8] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “Conflict-Free Replicated Data Types,” *Stabilization, Safety, and Security of Distributed Systems*, vol. 6976. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 386–400, 2011. doi: [10.1007/978-3-642-24550-3_29](https://doi.org/10.1007/978-3-642-24550-3_29).
- [9] M. Köhler, G. Zakhour, P. Weisenburger, and G. Salvaneschi, “Consistent Local-First Software: Enforcing Safety and Invariants for Local-First Applications,” *IEEE Transactions on Software Engineering*, vol. 51, no. 1, pp. 53–65, Jan. 2025, doi: [10.1109/TSE.2024.3477723](https://doi.org/10.1109/TSE.2024.3477723).
- [10] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design Science in Information Systems Research,” *MIS Q.*, vol. 28, no. 1, pp. 75–105, Mar. 2004.
- [11] K. Peffers, T. Tuunanen, M. Rothenberger, and S. Chatterjee, “A Design Science Research Methodology for Information Systems Research,” *Journal of Management Information Systems*, vol. 24, pp. 45–77, Jan. 2007.

- [12] J. Haas, R. Mogk, E. Yanakieva, A. Bieniusa, and M. Mezini, “LoRe: A Programming Model for Verifiably Safe Local-first Software,” *ACM Trans. Program. Lang. Syst.*, vol. 46, no. 1, pp. 2:1–2:26, Jan. 2024, doi: [10.1145/3633769](https://doi.org/10.1145/3633769).
- [13] G. DeCandia *et al.*, “Dynamo: Amazon's Highly Available Key-Value Store,” *SIGOPS Oper. Syst. Rev.*, vol. 41, no. 6, pp. 205–220, Oct. 2007, doi: [10.1145/1323293.1294281](https://doi.org/10.1145/1323293.1294281).
- [14] P. Bailis, A. Fekete, M. J. Franklin, A. Ghodsi, J. M. Hellerstein, and I. Stoica, “Coordination Avoidance in Database Systems,” *Proc. VLDB Endow.*, vol. 8, no. 3, pp. 185–196, Nov. 2014, doi: [10.14778/2735508.2735509](https://doi.org/10.14778/2735508.2735509).
- [15] P. E. O'Neil, “The Escrow Transactional Method,” *ACM Trans. Database Syst.*, vol. 11, no. 4, pp. 405–430, Dec. 1986, doi: [10.1145/7239.7265](https://doi.org/10.1145/7239.7265).
- [16] V. Balesgas *et al.*, “Putting Consistency Back into Eventual Consistency,” in *Proceedings of the Tenth European Conference on Computer Systems*, in EuroSys '15. New York, NY, USA: Association for Computing Machinery, Apr. 2015, pp. 1–16. doi: [10.1145/2741948.2741972](https://doi.org/10.1145/2741948.2741972).
- [17] N. Schiefer, G. Litt, and D. Jackson, “Merge What You Can, Fork What You Can't: Managing Data Integrity in Local-First Software,” in *Proceedings of the 9th Workshop on Principles and Practice of Consistency for Distributed Data*, in PaPoC '22. New York, NY, USA: Association for Computing Machinery, Apr. 2022, pp. 24–32. doi: [10.1145/3517209.3524041](https://doi.org/10.1145/3517209.3524041).
- [18] T. Renaux, S. Van den Vonder, and W. De Meuter, “Secure RDTs: Enforcing Access Control Policies for Offline Available JSON Data,” *Secure RDTs: Enforcing Access Control Policies for Offline Available JSON Data (Artifact)*, vol. 7, no. OOPSLA2, pp. 227:146–227:172, Oct. 2023, doi: [10.1145/3622802](https://doi.org/10.1145/3622802).
- [19] S. Gilbert and N. Lynch, “Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,” *SIGACT News*, vol. 33, no. 2, pp. 51–59, June 2002, doi: [10.1145/564585.564601](https://doi.org/10.1145/564585.564601).
- [20] P. S. Almeida, “Approaches to Conflict-free Replicated Data Types,” *ACM Computing Surveys*, vol. 57, no. 2, pp. 1–36, Feb. 2025, doi: [10.1145/3695249](https://doi.org/10.1145/3695249).
- [21] “What's a Conflicted Copy?.” Accessed: Feb. 16, 2026. [Online]. Available: <https://help.dropbox.com/organize/conflicted-copy>
- [22] J. Vepsäläinen, A. Hellas, and P. Vuorimaa, “Overview of Web Application Performance Optimization Techniques,” in *Web Information Systems and Technologies*, M. Marchiori and F. García Peñalvo, Eds., Cham: Springer Nature Switzerland, 2025, pp. 45–62. doi: [10.1007/978-3-031-89621-7_3](https://doi.org/10.1007/978-3-031-89621-7_3).
- [23] “Google's Vint Cerf Warns of 'Digital Dark Age',” *BBC News*, Feb. 2015, Accessed: Feb. 01, 2026. [Online]. Available: <https://www.bbc.com/news/science-environment-31450389>
- [24] M. Ahamad, G. Neiger, J. E. Burns, P. Kohli, and P. W. Hutto, “Causal Memory: Definitions, Implementation, and Programming,” *Distributed Computing*, vol. 9, no. 1, pp. 37–49, Mar. 1995, doi: [10.1007/BF01784241](https://doi.org/10.1007/BF01784241).
- [25] Y. Saito and M. Shapiro, “Optimistic Replication,” *ACM Comput. Surv.*, vol. 37, no. 1, pp. 42–81, Mar. 2005, doi: [10.1145/1057977.1057980](https://doi.org/10.1145/1057977.1057980).

- [26] H. Garcia-Molina and K. Salem, "Sagas," in *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, in SIGMOD '87. New York, NY, USA: Association for Computing Machinery, Dec. 1987, pp. 249–259. doi: [10.1145/38713.38742](https://doi.org/10.1145/38713.38742).
- [27] B. Moseley and P. Marks, "Out of the Tar Pit," *Software Practice Advancement (SPA)*, vol. 2006, 2006.
- [28] R. Abdullin, "CQRS Documents by Greg Young."
- [29] M. Fowler, "Bliki: CQRS." Accessed: Mar. 20, 2026. [Online]. Available: <https://martinfowler.com/bliki/CQRS.html>
- [30] "2019_4740 Compliance with SCA in Offline Mode on an Aircraft without Internet Connection | European Banking Authority." Accessed: Mar. 13, 2026. [Online]. Available: https://www.eba.europa.eu/single-rule-book-qa/qna/view/publicId/2019_4740
- [31] "Directive (EU) 2015/2366 of the European Parliament and of the Council of 25 November 2015 on Payment Services in the Internal Market, Amending Directives 2002/65/EC, 2009/110/EC and 2013/36/EU and Regulation (EU) No 1093/2010, and Repealing Directive 2007/64/EC (Text with EEA Relevance)." Accessed: Mar. 13, 2026. [Online]. Available: <http://data.europa.eu/eli/dir/2015/2366/oj>
- [32] "Payment Card Industry Data Security Standard," 2024.
- [33] "Payment Tokenisation: What It Is and How It Works | Stripe." Accessed: Mar. 13, 2026. [Online]. Available: <https://stripe.com/en-de/resources/more/payment-tokenization-101>
- [34] Mastercard, "Authentication Guide for Europe." Accessed: Mar. 13, 2026. [Online]. Available: https://www.mastercard.de/content/dam/public/mastercardcom/eu/de/images/Heandler/ATNGE_Manual_v1.3.pdf
- [35] S. Gilbert and N. Lynch, "Perspectives on the CAP Theorem," *Computer*, vol. 45, no. 2, pp. 30–36, Feb. 2012, doi: [10.1109/MC.2011.389](https://doi.org/10.1109/MC.2011.389).
- [36] A. S. Asrese, S. J. Eravuchira, V. Bajpai, P. Sarolahti, and J. Ott, "Measuring Web Latency and Rendering Performance: Method, Tools, and Longitudinal Dataset," *IEEE Transactions on Network and Service Management*, vol. 16, no. 2, pp. 535–549, June 2019, doi: [10.1109/TNSM.2019.2896710](https://doi.org/10.1109/TNSM.2019.2896710).
- [37] B. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking Cloud Serving Systems with YCSB," in *ResearchGate*, doi: [10.1145/1807128.1807152](https://doi.org/10.1145/1807128.1807152).
- [38] A. Hevner, "A Three Cycle View of Design Science Research," *Scandinavian Journal of Information Systems*, vol. 19, Jan. 2007.
- [39] "Storage Quotas and Eviction Criteria - Web APIs | MDN." Accessed: Oct. 14, 2025. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Storage_API/Storage_quotas_and_eviction_criteria
- [40] "RxDB - JavaScript Database." Accessed: Mar. 18, 2026. [Online]. Available: <https://rxdb.info/>
- [41] "Electric | Data Platform for Multi-Agent." Accessed: Mar. 18, 2026. [Online]. Available: <https://electric-sql.com/>

- [42] P. G. D. Group, “PostgreSQL.” Accessed: Mar. 18, 2026. [Online]. Available: <https://www.postgresql.org/>
- [43] “Automerger.” Accessed: Mar. 18, 2026. [Online]. Available: <https://automerger.org/>
- [44] “Yjs | Homepage.” Accessed: Mar. 18, 2026. [Online]. Available: <https://yjs.dev/>
- [45] K. Jahns, “Dmonad/Crdt-Benchmarks.” Accessed: Mar. 25, 2026. [Online]. Available: <https://github.com/dmonad/crdt-benchmarks>
- [46] “gRPC.” Accessed: Mar. 18, 2026. [Online]. Available: <https://grpc.io/>
- [47] “GraphQL | A Query Language for Your API.” Accessed: Mar. 18, 2026. [Online]. Available: <https://graphql.org/>
- [48] “tRPC - Move Fast and Break Nothing. End-to-end Typesafe APIs Made Easy. | tRPC.” Accessed: Mar. 18, 2026. [Online]. Available: <https://trpc.io/>
- [49] “Drizzle ORM - next Gen TypeScript ORM..” Accessed: Mar. 18, 2026. [Online]. Available: <https://orm.drizzle.team/>
- [50] “TanStack Start.” Accessed: Mar. 18, 2026. [Online]. Available: <https://tanstack.com/start/latest>
- [51] “Fast and Reliable End-to-End Testing for Modern Web Apps | Playwright.” Accessed: Mar. 18, 2026. [Online]. Available: <https://playwright.dev/>
- [52] “Chrome DevTools Protocol.” Accessed: Mar. 07, 2026. [Online]. Available: <https://chromedevtools.github.io/devtools-protocol/>
- [53] “Fast and Reliable End-to-End Testing for Modern Web Apps | Playwright.” Accessed: Mar. 07, 2026. [Online]. Available: <https://playwright.dev/>
- [54] “Network Throttling in Chrome DevTools | DebugBear.” Accessed: Mar. 24, 2026. [Online]. Available: <https://www.debugbear.com/blog/chrome-devtools-network-throttling>
- [55] “TanStack Query.” Accessed: Mar. 18, 2026. [Online]. Available: <https://tanstack.com/query/latest>
- [56] “Vite.” Accessed: Mar. 18, 2026. [Online]. Available: <https://vite.dev/>
- [57] C. W. Server, “Caddy - The Ultimate Server with Automatic HTTPS.” Accessed: Mar. 18, 2026. [Online]. Available: <https://caddyserver.com/>
- [58] “The State of Shopify in 2026.” Accessed: Feb. 17, 2026. [Online]. Available: <https://storeleads.app/reports/shopify>
- [59] Montgomery, *Applied Statistics and Probability for Engineers*, 7E. John Wiley & Sons, Incorporated, 2017.
- [60] J. Dean and L. A. Barroso, “The Tail at Scale,” *Commun. ACM*, vol. 56, no. 2, pp. 74–80, Feb. 2013, doi: [10.1145/2408776.2408794](https://doi.org/10.1145/2408776.2408794).
- [61] I. Zavalvshyn, A. Legay, A. Rath, and E. Rivière, “Local-First Smart Home Applications with HubOS,” in *2025 20th European Dependable Computing Conference (EDCC)*, Apr. 2025, pp. 18–26. doi: [10.1109/EDCC66201.2025.00015](https://doi.org/10.1109/EDCC66201.2025.00015).

- [62] L. Siffre, T. Ledoux, R. Pawlak, and J. Guery, “Local Computing vs. Cloud Computing: An Empirical Study of Energy Consumption,” in *Proceedings of the 18th IEEE/ACM International Conference on Utility and Cloud Computing*, in UCC '25. New York, NY, USA: Association for Computing Machinery, Dec. 2025, pp. 1–10. doi: [10.1145/3773274.3774278](https://doi.org/10.1145/3773274.3774278).
- [63] “The Mobile Economy 2025.”
- [64] L. Siffre, T. Ledoux, R. Pawlak, and J. Guery, “Local-First Software for Green IT,” in *2025 11th International Conference on ICT for Sustainability (ICT4S)*, June 2025, pp. 58–68. doi: [10.1109/ICT4S68164.2025.00015](https://doi.org/10.1109/ICT4S68164.2025.00015).
- [65] “How to Avoid Loss of Receivables | Stripe.” Accessed: Mar. 13, 2026. [Online]. Available: <https://stripe.com/en-de/resources/more/loss-of-receivables-germany>
- [66] “Regulation (EU) 2024/886 of the European Parliament and of the Council of 13~March 2024 Amending Regulations (EU) No~260/2012 and (EU)~2021/1230 and Directives~98/26/EC and (EU)~2015/2366 as Regards Instant Credit Transfers in Euro (Text with EEA Relevance).” Accessed: Mar. 13, 2026. [Online]. Available: <http://data.europa.eu/eli/reg/2024/886/oj>
- [67] “Merchant Initiated Transactions (MITs): When to Categorize a Transaction as MIT : Stripe: Help & Support.” Accessed: Mar. 13, 2026. [Online]. Available: [https://support.stripe.com/questions/merchant-initiated-transactions-\(mits\)-when-to-categorize-a-transaction-as-mit](https://support.stripe.com/questions/merchant-initiated-transactions-(mits)-when-to-categorize-a-transaction-as-mit)
- [68] “2019_4776 More than One Transaction from a Single Consumer Initiated Transaction | European Banking Authority.” Accessed: Mar. 13, 2026. [Online]. Available: https://www.eba.europa.eu/single-rule-book-qa/qna/view/publicId/2019_4776
- [69] “Global Payments Report | Payment Insights | Worldpay.” Accessed: Mar. 13, 2026. [Online]. Available: <https://www.worldpay.com/en/global-payments-report>
- [70] “B2B Payment Practices Trends in Western Europe 2025.” Accessed: Mar. 13, 2026. [Online]. Available: <https://atradiuscollections.com/in/knowledge-and-research/reports/b2b-payment-practices-trends-western-europe-2025>
- [71] “Commission Delegated Regulation (EU) 2018/389 of 27 November 2017 Supplementing Directive (EU) 2015/2366 of the European Parliament and of the Council with Regard to Regulatory Technical Standards for Strong Customer Authentication and Common and Secure Open Standards of Communication (Text with EEA Relevance.).” Accessed: Mar. 13, 2026. [Online]. Available: http://data.europa.eu/eli/reg_del/2018/389/oj

Einsatz generativer KI

Bei der Erstellung der vorliegenden Arbeit wurde generative künstliche Intelligenz (KI) eingesetzt. Der Einsatz erfolgte unter kontinuierlich kritischer Prüfung und Überarbeitung der durch KI generierten Inhalte. Die wissenschaftliche und inhaltliche Verantwortung für die Arbeit liegt vollumfänglich beim Verfasser.

Folgende Modelle wurden für die genannten Zwecke genutzt:

Methodik und Forschungsdesign

Zur kritischen Prüfung und Evaluation der Forschungs- bzw. Leitfrage wurde das Modell Gemini 2.5 Pro herangezogen.

Literaturrecherche

Für eine erste, oberflächliche und breit angelegte Recherche des Themenfeldes sowie zur Gliederung von Inhalten wurden die Modelle Gemini 2.5 Flash, Gemini 2.5 Pro, Gemini 3.0 Flash sowie Gemini 3.0 Pro verwendet. Die daraus gewonnenen Informationen dienten ausschließlich als Ausgangspunkt und wurden im Anschluss durch eine methodische wissenschaftliche Literaturrecherche überprüft, validiert und vertieft.

Literaturanalyse

Für die Zusammenfassung und Erklärung ausgewählter, relevanter Literaturquellen wurde NotebookLM eingesetzt.

Software-Entwicklung

Im Rahmen des Praxisprojekts dieser Arbeit, der Entwicklung des Prototype, wurde GitHub Copilot zur Code-Vervollständigung eingesetzt. Die Modelle Gemini 2.5 Flash, Gemini 2.5 Pro, Gemini 3.0 Flash sowie Gemini 3.0 Pro wurden zudem für die konzeptionelle Unterstützung der Entwicklung genutzt. Der gesamte generierte Code wurde auf Funktionalität, Korrektheit und Sicherheit hin geprüft, getestet und bei Bedarf angepasst.

Textüberarbeitung

Für das Review von geschriebenen Textpassagen sowie für die sprachliche Umformulierung und Optimierung selbst verfasster Textentwürfe wurde Gemini 3.0 Flash und Gemini 3.0 Pro unterstützend eingesetzt.

Formatierung

Zur korrekten Formatierung und Verwendung von typst-Befehlen wurden Gemini 3.0 Flash und Gemini 3.0 Pro verwendet.

Raw Benchmark Data

A-A Initial Synchronization and Load Latency

Architecture	Profile	Dataset Size	Median	Mean	95th Pctl
Local-First	Baseline	100	2394.25 ms	2393.99 ms	2416.53 ms
Local-First	Baseline	5000	2353.81 ms	2361.01 ms	2411.75 ms
Local-First	Baseline	25000	4609.42 ms	4637.14 ms	5079.24 ms
Local-First	Budget	100	2734.36 ms	2998.24 ms	3806.6 ms
Local-First	Budget	5000	4172.74 ms	4264.5 ms	4839.96 ms
Local-First	Budget	25000	7554.94 ms	7690.27 ms	8775.02 ms
Local-First	Commuter	100	25891.96 ms	25901.59 ms	26014.27 ms
Local-First	Commuter	5000	26897.36 ms	26908.66 ms	27054.62 ms
Local-First	Commuter	25000	51707.71 ms	51628.03 ms	52332.96 ms
Local-First	Worst-Case	100	26298.04 ms	26562.91 ms	27287.24 ms
Local-First	Worst-Case	5000	28029.83 ms	27868.26 ms	28563.94 ms
Local-First	Worst-Case	25000	54032.5 ms	54162.63 ms	55202.24 ms
SSR-SPA	Baseline	100	1351.23 ms	1359.24 ms	1415.87 ms
SSR-SPA	Baseline	5000	1340.11 ms	1351.32 ms	1399.67 ms
SSR-SPA	Baseline	25000	1354.01 ms	1358.56 ms	1385.27 ms
SSR-SPA	Budget	100	1546.1 ms	1583.72 ms	1621.95 ms
SSR-SPA	Budget	5000	1545.22 ms	1539.84 ms	1592.33 ms
SSR-SPA	Budget	25000	1559.91 ms	1561.03 ms	1600.46 ms
SSR-SPA	Commuter	100	12682.83 ms	12678.72 ms	12717.23 ms
SSR-SPA	Commuter	5000	12663.23 ms	12663.62 ms	12699.85 ms
SSR-SPA	Commuter	25000	12669.13 ms	12671.53 ms	12733.39 ms
SSR-SPA	Worst-Case	100	13779.41 ms	13843.68 ms	14082.52 ms
SSR-SPA	Worst-Case	5000	13765.67 ms	13834.29 ms	14071.02 ms
SSR-SPA	Worst-Case	25000	13784.88 ms	13846.54 ms	14016.5 ms

A-B Interaction Latency (Search & Filter)

Architecture	Profile	Dataset Size	Median	Mean	95th Pctl
Local-First	Baseline	100	21.85 ms	22.72 ms	28.51 ms
Local-First	Baseline	5000	53.25 ms	56.89 ms	75.96 ms
Local-First	Baseline	25000	192.55 ms	197.03 ms	254.93 ms

Architecture	Profile	Dataset Size	Median	Mean	95th Pctl
Local-First	Budget	100	87.7 ms	87.7 ms	101.25 ms
Local-First	Budget	5000	205.3 ms	217.51 ms	287.7 ms
Local-First	Budget	25000	613.4 ms	640.27 ms	819.84 ms
Local-First	Commuter	100	22.15 ms	23.13 ms	27.05 ms
Local-First	Commuter	5000	61.5 ms	64.73 ms	83.92 ms
Local-First	Commuter	25000	197.95 ms	209.82 ms	282.41 ms
Local-First	Worst-Case	100	83.55 ms	83.45 ms	90.47 ms
Local-First	Worst-Case	5000	199.65 ms	207.78 ms	258.84 ms
Local-First	Worst-Case	25000	665.85 ms	687.54 ms	884.74 ms
SSR-SPA	Baseline	100	201.55 ms	201.95 ms	209.37 ms
SSR-SPA	Baseline	5000	205.85 ms	205.74 ms	213.22 ms
SSR-SPA	Baseline	25000	205.35 ms	206.06 ms	214.06 ms
SSR-SPA	Budget	100	246.95 ms	246.27 ms	259.53 ms
SSR-SPA	Budget	5000	248.2 ms	248.2 ms	258.74 ms
SSR-SPA	Budget	25000	244.85 ms	245.02 ms	256.89 ms
SSR-SPA	Commuter	100	2213.3 ms	2207.61 ms	2256.41 ms
SSR-SPA	Commuter	5000	2213.85 ms	2210.61 ms	2261.72 ms
SSR-SPA	Commuter	25000	2220.6 ms	2209.41 ms	2258.36 ms
SSR-SPA	Worst-Case	100	2128.3 ms	2129.56 ms	2149.82 ms
SSR-SPA	Worst-Case	5000	2120.4 ms	2126.1 ms	2151.58 ms
SSR-SPA	Worst-Case	25000	2124.15 ms	2125.59 ms	2149.58 ms

A-C Browser Storage Footprint

Architecture	Profile	Dataset Size	Median	Mean	95th Pctl
Local-First	Baseline	100	3.57 MB	3.57 MB	3.57 MB
Local-First	Baseline	5000	4.6 MB	4.6 MB	4.6 MB
Local-First	Baseline	25000	8.81 MB	8.81 MB	8.81 MB
Local-First	Budget	100	3.57 MB	3.57 MB	3.57 MB
Local-First	Budget	5000	4.6 MB	4.67 MB	4.6 MB
Local-First	Budget	25000	8.81 MB	8.81 MB	8.81 MB
Local-First	Commuter	100	3.57 MB	3.57 MB	3.57 MB
Local-First	Commuter	5000	4.6 MB	4.6 MB	4.6 MB
Local-First	Commuter	25000	8.81 MB	8.81 MB	8.81 MB
Local-First	Worst-Case	100	3.57 MB	3.57 MB	3.57 MB
Local-First	Worst-Case	5000	4.6 MB	4.6 MB	4.6 MB
Local-First	Worst-Case	25000	8.81 MB	8.81 MB	8.81 MB
SSR-SPA	Baseline	100	0 MB	0 MB	0 MB

Architecture	Profile	Dataset Size	Median	Mean	95th Pctl
SSR-SPA	Baseline	5000	0 MB	0 MB	0 MB
SSR-SPA	Baseline	25000	0 MB	0 MB	0 MB
SSR-SPA	Budget	100	0 MB	0 MB	0 MB
SSR-SPA	Budget	5000	0 MB	0 MB	0 MB
SSR-SPA	Budget	25000	0 MB	0 MB	0 MB
SSR-SPA	Commuter	100	0 MB	0 MB	0 MB
SSR-SPA	Commuter	5000	0 MB	0 MB	0 MB
SSR-SPA	Commuter	25000	0 MB	0 MB	0 MB
SSR-SPA	Worst-Case	100	0 MB	0 MB	0 MB
SSR-SPA	Worst-Case	5000	0 MB	0 MB	0 MB
SSR-SPA	Worst-Case	25000	0 MB	0 MB	0 MB

A-D Client Memory Tax (JS Heap)

Architecture	Profile	Dataset Size	Median	Mean	95th Pctl
Local-First	Baseline	100	20.15 MB	17.06 MB	20.83 MB
Local-First	Baseline	5000	22.84 MB	23.14 MB	23.53 MB
Local-First	Baseline	25000	99.37 MB	98.52 MB	104.32 MB
Local-First	Budget	100	18.79 MB	18.71 MB	20.21 MB
Local-First	Budget	5000	24.01 MB	24.06 MB	24.52 MB
Local-First	Budget	25000	116.73 MB	113.46 MB	117.47 MB
Local-First	Commuter	100	12 MB	12.02 MB	12.31 MB
Local-First	Commuter	5000	34.35 MB	32.52 MB	34.96 MB
Local-First	Commuter	25000	70.13 MB	67.02 MB	88.67 MB
Local-First	Worst-Case	100	11.83 MB	12.91 MB	19.17 MB
Local-First	Worst-Case	5000	22.52 MB	24.05 MB	38.36 MB
Local-First	Worst-Case	25000	71.77 MB	75.32 MB	92.95 MB
SSR-SPA	Baseline	100	15.44 MB	15.49 MB	15.81 MB
SSR-SPA	Baseline	5000	15.46 MB	15.45 MB	15.51 MB
SSR-SPA	Baseline	25000	15.44 MB	15.44 MB	15.51 MB
SSR-SPA	Budget	100	14.69 MB	14.83 MB	15.37 MB
SSR-SPA	Budget	5000	14.65 MB	14.8 MB	15.31 MB
SSR-SPA	Budget	25000	14.63 MB	14.74 MB	15.22 MB
SSR-SPA	Commuter	100	14.62 MB	14.54 MB	14.7 MB
SSR-SPA	Commuter	5000	14.64 MB	14.56 MB	14.7 MB
SSR-SPA	Commuter	25000	14.64 MB	14.56 MB	14.73 MB
SSR-SPA	Worst-Case	100	15.02 MB	15.02 MB	15.06 MB
SSR-SPA	Worst-Case	5000	15.02 MB	15.02 MB	15.08 MB

Architecture	Profile	Dataset Size	Median	Mean	95th Pctl
SSR-SPA	Worst-Case	25000	15.02 MB	15.02 MB	15.06 MB

Implementation Details

B-A Solving Schema Rigidity with Custom Fields

A major challenge in local-first **Structured Query Language** (SQL) environments is schema rigidity. To allow for flexible product attributes without requiring constant database migrations (which are difficult to propagate to offline clients), the implementation introduces an **Entity-Attribute-Value** (EAV) pattern within the strict relational schema.

The tables `custom_field_definitions` and `custom_field_values` allow products to possess dynamic, typed attributes (Text, Number, Date, Boolean). The client-side application logic reconstructs these into usable objects at runtime. This hybrid approach combines the integrity of SQL with the flexibility typically associated with NoSQL document stores.

B-B Sorting via Fractional Indexing

A specific technical challenge was implementing drag-and-drop reordering in a collaborative environment without index conflicts. If User A and User B both move an item to index 0, a conflict occurs.

The implementation solves this using fractional indexing. When an item is moved between two others, a new sorting key is generated that is lexicographically halfway between the neighbors (e.g., between “a” and “b” is “an”). This allows the list to be reordered infinitely by concurrent users without ever requiring a global re-indexing operation or locking.

B-C Session Isolation and Reactivity

To support multiple distinct carts (e.g., “Personal” vs. “Work”), the implementation utilizes a specific React pattern to ensure isolation. The `CartSession` component is mounted with a unique key: `key={activeCartId}`. This forces the React lifecycle to completely destroy and re-initialize the Y.Doc when switching contexts, preventing data leaks between distinct collaborative sessions.

The bridge between the imperative Yjs model and the declarative React UI is established via an `observeDeep` listener. This listener detects incoming operations from remote users and triggers a local state update, converting the Y.Map into a plain JavaScript array for rendering.

B-D Awareness and Presence

To fully realize the collaborative experience, the system must broadcast user presence, such as who is currently online or viewing the cart. Because this information is highly

ephemeral and does not need to be permanently stored in the database, it is not modeled within the core Yjs document (Y.Doc).

Instead, the implementation utilizes the `y-protocols/awareness` module. The awareness protocol is an accompanying CRDT mechanism specifically designed to handle volatile, session-based state. It operates on a timeout basis, meaning a user's state automatically vanishes if they disconnect or close the application.

In this architecture, each client injects their current user profile (including their user ID and name) into their local awareness state. The network provider (`y-electric`) then multiplexes these awareness updates alongside standard document operations over the same PostgreSQL-backed ElectricSQL replication stream.

On the receiving end, a React listener observes these awareness changes. It dynamically aggregates the active states, deduplicates them by user ID, and surfaces an array of `onlineUsers` to the application. This drives real-time UI features, such as rendering active avatars, without polluting the persistent document history or requiring complex database cleanup for transient connection events.

B-E Eliminating Render Flicker via Route Pre-loading

A primary UX objective of the local-first architecture is the “No Spinners” ideal: the complete elimination of loading indicators during standard application navigation. While the reactive query engine provides low-latency data access, simply mounting a React component that triggers a local database read can still yield a brief, single-frame render of an empty state (loading flicker) while the query resolves.

To bridge this gap and achieve true zero-latency navigation, the implementation integrates TanStack Router. This router supports asynchronous `loader` functions that execute before a route begins rendering. The application leverages this feature to eagerly preload the required TanStack DB collections into memory.

```
// Source: TanStack Router loader preventing render flicker
export const Route = createFileRoute("/orders")({
  loader: async () => {
    // Blocks route rendering until local data is primed
    await Promise.all([ordersCollection.preload()]);
  }
});
```

By invoking `ordersCollection.preload()` within the routing lifecycle, the data is guaranteed to be primed in the local reactive cache before the UI layer mounts. Consequently, when the React component renders and calls `useLiveQuery`, the local data is immediately yielded synchronously. This orchestration between the file-based router and the embedded client database completely eliminates loading flicker, ensuring the UI feels native and instantaneously responsive.

B-F Self-Healing Orphaned Nodes

In collaborative nested structures, concurrent operations, such as one user adding an item to a folder while another user deletes that same folder, can result in orphaned data nodes. To prevent data loss and maintain structural integrity, the application implements a reactive self-healing mechanism.

The system continuously monitors the flattened tree structure. It maps all valid, existing node IDs and identifies any nodes whose `parent_id` references a deleted or non-existent element. When orphans are detected, a Yjs transaction is automatically dispatched to reset their `parent_id` to `null`. This rescues the orphaned items by elevating them to the root level of the hierarchy, ensuring they are safely recovered without requiring manual intervention.

```
// Source: React hook for reparenting orphaned nodes
useEffect(() => {
  if (flatNodes.length === 0) return;

  const existingNodeIds = new Set(flatNodes.map((n) => n.id));
  const orphans = flatNodes.filter(
    (node) => node.parent_id !== null && !existingNodeIds.has(node.parent_id)
  );

  if (orphans.length > 0) {
    ydoc.transact(() => {
      orphans.forEach((orphan) => {
        const yNode = nodesMap.get(orphan.id);
        if (yNode) yNode.set("parent_id", null);
      });
    }, "self-heal-orphans");
  }
}, [flatNodes, ydoc, nodesMap]);
```